

Registration-Optimized Dynamic Group Time-based One-time Passwords for Mobile Access

Jiaqing Guo¹, Xuelian Cao², Zengpeng Li³, Yong Zhou¹, Zheng Yang^{1*}, and
Jianying Zhou⁴

¹ Southwest University, Chongqing, China

{whoami, hiro115}@email.swu.edu.cn, youngzheng@swu.edu.cn

² Tsinghua University, Beijing, China

xl-caoc@mail.tsinghua.edu.cn

³ Shandong University, Jinan, China

zengpeng@email.sdu.edu.cn

⁴ Singapore University of Technology and Design, Singapore

jianying_zhou@sutd.edu.sg

Abstract. Mobile access within public finance and enterprise environments often requires lightweight anonymous authentication, allowing users to prove authorization without disclosing their identities. Group Time-based One-Time Passwords (GTOTP) has recently been proposed as a lightweight primitive meeting this need with post-quantum security. To address dynamic group membership, Cao et al. introduced DGTOne, the first dynamic GTOTP construction. It employs chameleon hashes to precompute a fixed set of Merkle-tree leaves (mount points), into which conventional TOTP verification points (VPs) contributed by group members are adaptively inserted. However, DGTOne partitions mount points by time epochs, so they can expire and become unusable, causing capacity waste due to unpredictable join times. Moreover, its outsourced proof generation requires verifiers to be online each epoch to fetch refreshed credentials from Registration Authority (RA), defeating offline verification needed in mobile access. We address these limitations with two new schemes. First, we propose NWDGT, a no-wastage DGTOTP design that constructs Merkle trees of members' verification points (VP-trees) on demand, eliminating expired mount points at the cost of added handling latency. To mitigate this latency, we introduce LWDGT, which instantiates multiple small one-time signature (OTS) trees whose leaves (OTS public keys) serve as mount points. New members' VPs are signed immediately using unused leaves, achieving low wastage. We formally prove that the wastage rate of LWDGT is, with overwhelming probability, lower than that of DGTOne. By modeling the registration process and optimizing OTS-tree size, for deployments with up to 500 members (209 initially, 20 added monthly), LWDGT reduces mount point wastage rate by 10.2% over one year compared to DGTOne.

Keywords: Group Time-based One-Time Passwords · Anonymity · Traceability · Authentication.

* Corresponding author.

1 Introduction

Mobile access has become an indispensable component of modern infrastructures, supporting applications in finance and enterprise access control [9,28,29]. In scenarios like bank or stock exchange entry, authentication devices are typically resource-constrained smartcard readers or embedded terminals with limited storage, computational power, and connectivity. Meanwhile, these settings involve highly sensitive operations (e.g., financial transactions, and entertainment activities), where users prefer not to expose identities, making privacy-preserving authentication a fundamental requirement for mobile access systems [18]. Lightweight anonymous authentication thus emerges as a promising approach, enabling users to prove access rights without revealing identities.

To realize such lightweight anonymous authentication, Yang et al. [34] introduced a primitive named *Group Time-based One-Time Passwords* (GTOTP). Their construction, referred to as YJN+ scheme, extends asymmetric time-based one-time passwords (TOTP) into an anonymous group setting, allowing members to authenticate by proving group membership without revealing identities. Credentials are generated at registration and returned directly to members, avoiding outsourced generation by RA. By relying solely on hash functions, the scheme is not only lightweight and suitable for embedded devices, but also post-quantum secure. Nevertheless, this approach requires each member to have a sufficient auxiliary storage device (such as a mobile phone) to store Merkle proofs for all future epochs, each covering only a short validity window for verification. However, the YJN+ scheme assumes a static group model in which all members are predefined and immutable, making it unsuitable for dynamic settings where participants join and leave the group adaptively at will.

To enable dynamic group management (DGM), Cao et al. [10] recently proposed an enhanced primitive *Dynamic Group Time-based One-Time Passwords* (DGTOTP). Their concrete construction, called DGTOne scheme, extends asymmetric TOTP using chameleon hash functions and a Merkle tree whose root acts as the group public key. It pre-generates leaves as *mount points* using chameleon hash functions. These mount points can later be used to register new members, allowing them to join without affecting existing ones. Unlike YJN+, however, DGTOne outsources credential generation to the registration authority (RA), requiring the verifier to fetch updated proofs from the RA in each epoch. Consequently, a DGTOne verifier must remain online at all times, which limits its applicability in scenarios where verifier devices have intermittent or restricted network access. For instance, many access devices—such as smartcard readers at physical entry gates—are designed to operate with unreliable or limited connectivity due to security isolation concerns.

Moreover, DGTOne partitions mount points by epochs, requiring a fixed number to be pre-generated for each epoch. These mount points can expire and become unusable, leading to a high wastage rate (i.e., the fraction of unused mount points over the total provided) when member join times are unpredictable. To support offline verification, credentials cannot be outsourced to RA but must instead be stored directly by members, as in YJN+ scheme. In this case, the

total number of mount points available for member registration is fixed, making it important to maximize their utilization so that the verifier can authenticate as many members as possible, thereby reducing the wastage rate. In summary, this raises the key research question:

“Can we construct a non-outsourced DGTOTP scheme with lower wastage rate for mobile access?”

Our Works. To further explore this question, we illustrate in Figure 1 a mobile access scenario where a member authenticates at the access gate using a password stored on a mobile phone. The gate acts as the verifier device, which may have intermittent or limited network connectivity due to operational constraints or security isolation. The verifier device is periodically updated by an entity responsible for its maintenance, referred to as the *administrator of the verifier* (abbreviated as *admin of verifier* in Figure 1). This entity can be a human operator, an automated management system, or another trusted component with network access, performing manual updates and maintenance tasks when connectivity is available but without continuous real-time synchronization. A member submits a registration request to RA, after which RA publishes the corresponding authentication information on a public bulletin board (PBB) that the administrator can later retrieve and update to the verifier device manually. The administrator can also download logs such as authentication records, but may be curious about the identities of authenticating members, making privacy preservation essential. After registration, a member can authenticate by transmitting the stored password through a secure channel, such as Near-Field Communication (NFC), to the verifier, while an adversary may attempt to forge passwords in order to gain unauthorized access.

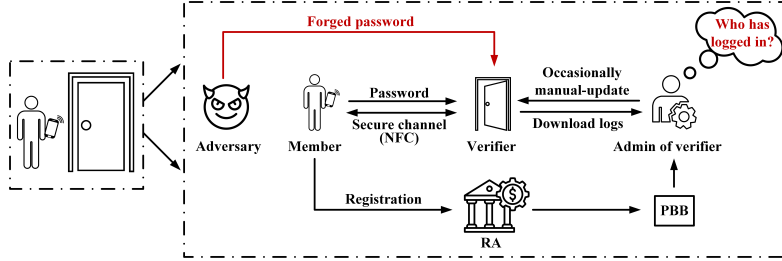


Fig. 1: System model illustration.

Building on this scenario, we propose two new DGTOTP constructions designed for different deployment environments, since anonymous authentication systems differ in their tolerance to registration-handling latency. One targets the **delayed-batch registration setting**, where short delays in registration are acceptable. This is typical in enterprise infrastructures, for example when employees are registered in advance before starting work or when new members are added at scheduled times. The other aims at **instant-response registration setting**, where registration must take effect immediately. Such a setting arises in

mobile and IoT environments, for instance when a visitor requests temporary access to a facility or when a new edge device joins a network and must be authenticated without delay.

Our first construction, a no-wastage DGTOTP scheme **NWDGT**, is designed for the delayed-batch registration setting. Designing such a scheme to achieve offline verification and eliminate mount point wastage presents several challenges. Since the group public key (gpk) is generated at system initialization, it is necessary to ensure that dynamically adding sub-trees for future epochs does not affect the existing gpk or the states of registered members. Achieving anonymity and traceability under adaptive adversaries further complicates the design.

To address these challenges, we customize a register-and-build-later (RBL) strategy. For each epoch, verification points are aggregated to construct a VP-tree before the epoch begins, thereby avoiding unused mount points. Since VP-trees depend on the actual registered members per epoch, they cannot be pre-generated, thus the gpk cannot be fixed at setup. To support this dynamic structure efficiently, we use a hash-based hyper-tree from SPHINCS+C [19], with the hyper-tree root as the gpk and each VP-tree mounted under it. Offline authentication is achieved by delivering all necessary credentials to members.

However, the waiting requirement in **NWDGT** is impractical for latency-sensitive scenarios, motivating our second construction, a low-wastage DGTOTP scheme **LWDGT**. It is designed for the instant-response registration setting, where immediate handling is required. Designing such a scheme introduces new challenges. First, RA must determine how to construct sub-trees efficiently during registration without waiting for future members to prevent handling delays. In addition, if we follow the sequential mounting strategy used in **NWDGT**, the order of sub-trees may reveal a member's joining time, leading to privacy leakage.

To resolve these issues, we develop a partitioned-build-on-demand (PBOD) registration strategy. Instead of building a single VP-tree for each epoch based on the registered verification points, we partition registrations into multiple smaller trees whose leaves serve as mount points. These trees, referred to as OTS-trees, are constructed using one-time signature (OTS) when needed. Each member's verification point is immediately signed by one of these leaves upon registration. To prevent the registration order from being inferred, we adopt the full SPHINCS+C construction, using its root as gpk. All OTS-trees are then signed and mounted under this structure, ensuring that the mounting order is unlinkable and that no information about member joining timing is exposed.

Contributions. The main contributions are as follows:

- We propose a no-wastage DGTOTP scheme, **NWDGT**, designed for the delayed-batch registration setting. Utilizing the RBL strategy, it constructs VP-trees based on actual registrations in each epoch. This eliminates mount point wastage, and it supports offline verification by providing credentials to members during registration. The trade-off is that RA and members must wait until the batch is complete, introducing registration handling latency.
- We develop a low-wastage DGTOTP scheme, **LWDGT**, tailored for the instant-response registration setting. Leveraging a PBOD strategy, it constructs small

OTS-trees on demand to enable fast registration handling and support offline verifiers by issuing the required credentials at registration time. While on-demand tree generation inevitably introduces some mount point wastage, our approach minimizes this overhead. Compared to NWDGT, LWDGT offers faster registration at the cost of slightly increased wastage.

- We provide a security analysis showing our design is secure. Through theoretical modeling of mount point wastage under realistic settings (systems with up to 500 users, 209 initial registrations, and 20 monthly arrivals), we demonstrate that LWDGT reduces wastage rate by 10.2% over one year compared to DGTOne scheme. In addition, our experimental evaluation confirms that both schemes remain practical and efficient in resource-constrained environments.

2 Related Work

The concept of One-Time Passwords (OTP) originated with Lamport’s seminal proposal [22], ensuring each password is valid for a single transaction. Building on this foundation, Sun et al. [30] proposed TrustOTP, a secure OTP solution to implement OTP schemes on the smartphone platform even in the presence of a malicious mobile OS. To further strengthen OTP-based authentication, Tolosana et al. [31] integrated handwritten digit biometrics on touchscreens, providing improved resistance against imitation attacks through behavioral analysis. In 2005, RFC 4226 standardized the HMAC-based One-Time Password (HOTP) algorithm [25], which uses a shared secret and synchronized counters to generate human-readable OTPs but risks desynchronization. RFC 6238 then proposed Time-based One-Time Password (TOTP) [24], replacing counters with timestamps to constrain OTP validity to short intervals. However, symmetric key reliance still exposes servers to compromise; asymmetric variants such as T/Key [20] enable public verification without server-stored seeds but lack anonymity, limiting their use in privacy-sensitive scenarios.

Group signature schemes [1,14,17,21] offer an alternative primitive for anonymous authentication [23,33,35,36]. In these schemes, users can produce signatures on behalf of a group without revealing their identity, while a designated group manager retains the ability to trace the signer if necessary. The concept was first introduced by Chaum and van Heyst [11], and formalized by Bellare et al. [2], who consolidated earlier informal notions into two key security properties: full anonymity and full traceability. This foundational model, now widely adopted, assumes a static group setting in which the list of group members is fixed at initialization. Building on this foundation, various efficient group signature schemes have been proposed [1,4,5,6,12], many of which follow Bellare et al.’s model. However, the original static model lacks support for member dynamics, which is critical in real-world applications where members frequently join or leave the group. To address this, Bellare et al. [4] proposed a formal security model for dynamic group signatures, which strengthens the traditional model to allow dynamic enrollment of members. In this model, the role of the group manager

is split between a group manager, responsible for managing membership, and a tracing manager, responsible for opening the signature when needed.

Although these schemes provide strong theoretical guarantees, they typically depend on expensive operations [7,15] (such as bilinear pairings or RSA accumulators) and incur complex, resource-intensive revocation processes. Consequently, they are ill-suited to settings with limited computational capacity or frequent user turnover. By contrast, GTOTP-style constructions, including those introduced in this work, achieve comparable security properties with markedly lower overhead, rendering them far more practical for deployment on resource-constrained devices. Compared to traditional TOTP schemes, GTOTP further realizes privacy by supporting anonymity, and as hash-based constructions, they avoid the computational costs inherent in group signatures. These advantages make DGTOTP the most practical and lightweight approach to anonymous authentication in real-world dynamic environments.

3 Preliminaries

We denote the security parameter by κ , and the set of integers between 1 and n by $[n] = \{1, \dots, n\} \subset \mathbb{N}$. Let $x \xleftarrow{\$} X$ be the operation of sampling x uniformly at random from the set X . Let $\parallel$ be the string concatenation operation.

Our constructions rely on several cryptographic primitives. We use (1) $\text{TOTP} := (\text{Setup}, \text{Plnit}, \text{PGen}, \text{Verify})$ to denote an asymmetric TOTP scheme; (2) $\text{PRF} : \mathcal{K}_{\text{PRF}} \times \mathcal{M}_{\text{PRF}} \rightarrow \mathcal{R}_{\text{PRF}}$ to denote a pseudo-random function family, where $\mathcal{K}_{\text{PRF}}$ is the key space, $\mathcal{M}_{\text{PRF}}$ is the message space, and $\mathcal{R}_{\text{PRF}}$ is the range space; (3) $\text{ASE} := (\text{Setup}, \text{Enc}, \text{Dec})$ to denote an authenticated symmetric encryption scheme; (4) $\text{CRH} : \mathcal{K}_{\text{CRH}} \times \mathcal{M}_{\text{CRH}} \rightarrow \mathcal{R}_{\text{CRH}}$ to denote a collision-resistant hash function, where $\mathcal{K}_{\text{CRH}}$ is the key space, $\mathcal{M}_{\text{CRH}}$ is the message space, and $\mathcal{R}_{\text{CRH}}$ is the range space; (5) $\text{MT} := (\text{Build}, \text{GetPf}, \text{Verify})$ to denote a Merkle tree scheme; (6) $\text{PM} := (\text{Setup}, \text{Shuffle})$ to denote an unpredictable permutation scheme; (7) $\text{Sig} := (\text{Setup}, \text{Sign}, \text{Verify})$ to denote a digital signature scheme. Their detailed definitions are described in Appendix A.

4 Security Notions

This section presents the security notions of our constructions, following [10].

Entities. The system involves three entities: group members, verifiers, and a registration authority (RA). Group members can generate valid time-based one-time passwords for authentication. RA performs system setup and manages member enrollment, where each member is uniquely identified by an identity ID.

Syntax. There are eight algorithms to describe in detail below.

- $(\text{pms}, \text{gpk}_G, \text{Gl}_G, \text{RL}_G, k_{\text{RA}}) \leftarrow \text{Setup}(1^\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e)$: RA runs this system setup algorithm. It takes the security parameter 1^κ , the group G , the password generation interval Δ_{pw} , the validity period of each verification point Δ_{vp} , and

the instance start and end time T_s and T_e . This algorithm outputs system parameters pms , the initial group public key gpk_G of a group G , the group management information Gl_G , the revocation list RL_G , and a secret key $k_{\text{RA}} \xleftarrow{\$} \mathcal{K}_{\text{RA}}$ for RA, where $\mathcal{K}_{\text{RA}}$ is the key space of RA.

- $(k_{\text{ID}_j}, \text{vps}_{\text{ID}_j}) \leftarrow \text{MInit}(\text{ID}_j)$: A group member runs this member initialization algorithm. It takes the identity of a member ID_j as input, and outputs the secret key $k_{\text{ID}_j} \xleftarrow{\$} \mathcal{K}_M$ and the verification state (i.e., a set of verification points) vps_{ID_j} belonging to member ID_j , where $\mathcal{K}_M$ is the secret key space of a group member.
- $(\text{Ax}_{\text{ID}_j}, \text{gpk}'_G, \text{Gl}'_G) \leftarrow \text{Join}(k_{\text{RA}}, \text{gpk}_G, \text{ID}_j, \text{vps}_{\text{ID}_j})$: RA runs this join algorithm to handle the enrollment of a member ID_j . It takes as input its secret key k_{RA} , group public key gpk_G , identity ID_j , and verification state vps_{ID_j} of the member. It outputs the auxiliary information Ax_{ID_j} as registration receipt to ID_j , and updates group management message Gl'_G and group public key gpk'_G if necessary.
- $sd^i_{\text{ID}_j} \leftarrow \text{GetSD}(k_{\text{ID}_j}, T)$: A group member runs this seed generation algorithm. It takes secret key $k_{\text{ID}_j} \in \mathcal{K}_M$ and time T , and outputs the secret seed $sd^i_{\text{ID}_j} \in \mathcal{S}_{\text{SD}}$ of the i -th verify epoch w.r.t. the time T , where $\mathcal{S}_{\text{SD}}$ is the secret seed space.
- $pw^{i,z}_{\text{ID}_j} \leftarrow \text{PwdGen}(sd^i_{\text{ID}_j}, T)$: A group member runs this password generation algorithm. It takes secret seed $sd^i_{\text{ID}_j}$ and time T as input, and outputs password $pw^{i,z}_{\text{ID}_j}$ the z -th password of the i -th verify epoch for member ID_j .
- $\{0, 1\} \leftarrow \text{Verify}(\text{gpk}_G, pw^{i,z}_{\text{ID}_j}, T, \text{RL}_G)$: Verifier runs this verification algorithm. It takes the group public key gpk_G , the password $pw^{i,z}_{\text{ID}_j}$, the time T , and the revocation list RL_G as input. It outputs 1 if $pw^{i,z}_{\text{ID}_j}$ is accepted, and 0 otherwise.
- $\text{RL}'_G \leftarrow \text{Revoke}(k_{\text{RA}}, \text{ID}_j, \text{gpk}_G, \text{RL}_G, \text{Gl}_G)$: RA runs this revocation algorithm to revoke the credentials of a member ID_j with respect to gpk_G by using its secret key k_{RA} , the revoking identity ID_j , and the group management message Gl_G . It returns the updated revocation list RL'_G .
- $\text{ID}_j \leftarrow \text{IDOpen}(k_{\text{RA}}, \text{gpk}_G, pw^{i,z}_{\text{ID}_j}, T)$: RA runs this identity extraction algorithm. It takes as input the secret key k_{RA} , group public key gpk_G , password $pw^{i,z}_{\text{ID}_j}$ and time T . It outputs ID_j if algorithm procedure is successful, and $\perp$ otherwise.

Threat Model. We assume RA is a non-colluding and semi-honest third party. RA will not be corrupted by any attackers and will faithfully register for members and trace the identity of malicious members, but may be curious about the status of password usage. The members can be corrupted by the attackers, but at least two members are honest. The attackers can control the network and thus can intercept, inject, and tamper with the communication between a member and a verifier. However, the communication between a member and RA is secure. The verifier can be malicious and may be curious about the identity of a member. The malicious group members and verifiers may infer the private information of honest group members.

Security Goals. To meet the security requirements of member authentication in group setting, the proposed schemes should meet the following security goals:

- *Anonymity*: Anonymity ensures that an adversary cannot infer any information about the real identity of an uncorrupted group member from an unused password even if the communication message is captured.
- *Traceability*: Only valid proofs can be traced to honest group members. This ensures that an adversary cannot forge a valid proof that can be attributed to either (i) an uncorrupted member of the group or (ii) any non-member.

Security Definition. Following [10], we define the correctness (Corr) and security properties of DGTOTP within a unified game-based framework [3]. The security properties include *anonymity* (Anony) and *traceability* (Trace)⁵. Each property is formalized as a game indexed by $Gvar \in \{\text{Corr}, \text{Anony}, \text{Trace}\}$. A challenger simulates the game procedures, illustrated in Figure 2. An adversary begins the game with `Initialize` and concludes with `Finalize`, issuing adaptive queries in between to achieve the winning condition $\text{Finalize}(b^*, pw^*, T^*) = 1$ for the corresponding game.

To model attacks on the join process, an adversary may adaptively introduce new members at any point using either `AddHM` (honest) or `AddMM` (malicious). Malicious members are fully controlled by the adversary, capturing insider threats, while their verification states vst_{ID_j} are generated by the adversary, and the challenger does not know their secrets. For honest members, the adversary may reveal long-term keys and secret seeds via `Corrupt` and `CompromiseSD` queries, respectively, simulating real-world compromises such as insider leaks, weak key management, or cryptanalysis.

Password generation and verification by honest members are modeled through `GetNextPw` and `ReceivePw`. The `GetNextPw` query captures known-password attacks without affecting unexposed secrets, while `ReceivePw` allows the adversary to test forged passwords, reflecting impersonation attempts. Membership revocation is modeled via `RevokeID`, enabling the adversary to manipulate group structure and infer information about honest members. Additionally, the `OpenID` procedure allows the adversary to reveal the identities associated with certain passwords, emphasizing that disclosure of one password must not compromise others.

Anonymity requires that the adversary cannot distinguish the secret seeds of two honest members in the `Challenge` procedure. Traceability requires that the adversary cannot produce a valid password-time pair (pw^*, T^*) satisfying either: (i) it is linked to an identity (including $\perp$) never added via `AddHM` or `AddMM`, or (ii) it belongs to an honest, uncorrupted member (whose key or secret seed was not exposed) and was never generated by the challenger during any procedure.

Definition 1. We say that a DGTOTP scheme Σ is correct if for any PPT adversary $\mathcal{A}$ and system parameters $Gpm = (\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e)$, $\Pr[G_{\mathcal{A}, \Sigma}^{\text{Corr}}(Gpm) = 0] \approx 1$, and we say that a correct scheme Σ is secure if for any PPT adversary $\mathcal{A}$ the advantages in the corresponding games are negligi-

⁵ Accordingly, adversaries are classified as either anonymity adversaries or traceability adversaries based on the property they aim to break.

ble, i.e., $\text{Adv}_{\mathcal{A}, \Sigma}^{\text{Anony}}(\text{Gpm}) := |\Pr[G_{\mathcal{A}, \Sigma}^{\text{Anony}}(\text{Gpm}) = 1] - 1/2|$ and $\text{Adv}_{\mathcal{A}, \Sigma}^{\text{Trace}}(\text{Gpm}) := \Pr[G_{\mathcal{A}, \Sigma}^{\text{Trace}}(\text{Gpm}) = 1]$.

$G_{\mathcal{A}, \Sigma}^{\text{Reg}}(rc, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e) :$ Initialize(): $(\text{pms}, \text{gpk}_G, \text{Gl}_G, \text{RL}_G, k_{RA}) \leftarrow \Sigma.\text{Setup}(1^\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e)$ Create lists $(\text{CML}, \text{CSL}, \text{HML}, \text{OL}) \leftarrow \emptyset$ Init a monotonically increasing (in Δ_{pw}) system clock $T_e := T_s$ OUTPUT $\text{pms}, \text{gpk}_G, \text{RL}_G$ Challenge(ID_0, ID_1): IF $(\text{ID}_0, \text{ID}_1) \notin \text{GP}$ or $\text{ID}_0 = \text{ID}_1$, OUTPUT $\perp$ $b \xleftarrow{\$} \{0, 1\}$ Switch to the next verification epoch $sd_{\text{ID}_0} \leftarrow \Sigma.\text{GetSD}(k_{\text{ID}_0}, T_e)$ Switch to the next verification epoch OUTPUT sd_{ID_0} Corrupt(ID_j): IF $\text{ID}_j \notin \text{HML}$, OUTPUT $\perp$ $\text{CML} \leftarrow \text{ID}_j$ OUTPUT k_{ID_j} CompromiseSD(ID_j): IF $\text{ID}_j \notin \text{HML}$, OUTPUT $\perp$ $sd_{\text{ID}_j} \leftarrow \Sigma.\text{GetSD}(k_{\text{ID}_j}, T_e)$ $\text{CSL} \leftarrow sd_{\text{ID}_j}$ OUTPUT sd_{ID_j} GetNextPw(): FOR $\forall \text{ID}_j \in \text{HML}$: $sd_{\text{ID}_j} \leftarrow \Sigma.\text{GetSD}(k_{\text{ID}_j}, T_e)$ $pw_{\text{ID}_j} \leftarrow \Sigma.\text{PwGen}(sd_{\text{ID}_j}, T_e)$ $\text{PL} \leftarrow \{pw_{\text{ID}_j} \text{ID}_j \in \text{HML}\}$ OUTPUT $\{pw_{\text{ID}_j} \text{ID}_j \in \text{HML}\}$	Finalize(b^*, pw^*, T^*): $\text{ID}^* := \Sigma.\text{IDOpen}(k_{RA}, \text{gpk}_G, pw^*, T^*)$ $vr := \Sigma.\text{Verify}(\text{gpk}_G, pw^*, T^*, \text{RL}_G)$ $i := \frac{T^* - T_s}{\Delta_{vp}}; T^* := T^* - ((T^* - T_s) \bmod \Delta_{pw})$ IF $\text{Gvar} = \text{Corr}$ and $pw^* \in \text{PL}$ and $(vr = 0 \vee \text{ID}^* \notin \text{HML})$ OUTPUT 1 IF $\text{Gvar} = \text{Anony}$ and $b = b^*$ and $\text{ID}_0 \notin (\text{CML} \cup \text{OL})$ and $\text{ID}_1 \notin (\text{CML} \cup \text{OL})$ OUTPUT 1 IF $\text{Gvar} = \text{Trace}$ and $vr = 1$ and $((\text{ID}^* \notin (\text{HML} \cup \text{CML})) \vee ((pw^*, T^*) \notin \text{PL} \wedge \text{ID}^* \notin \text{CML} \wedge sd_{\text{ID}^*} \notin \text{CSL}))$ OUTPUT 1 OUTPUT 0 AddHM(ID_j): $(k_{\text{ID}_j}, \text{vps}_{\text{ID}_j}) \leftarrow \text{Minit}(\text{ID}_j)$ $(\text{Ax}_{\text{ID}_j}, \text{gpk}_G, \text{Gl}_G) \leftarrow \Sigma.\text{Join}(k_{RA}, \text{gpk}_G, \text{ID}_j, \text{vps}_{\text{ID}_j})$ $\text{HML} \leftarrow \text{ID}_j$ OUTPUT $\text{Ax}_{\text{ID}_j}, \text{gpk}_G', \text{Gl}_G' \text{ID}_j$ AddMM($\text{ID}_j, \text{vps}_{\text{ID}_j}$): $(\text{Ax}_{\text{ID}_j}, \text{gpk}_G, \text{Gl}_G) \leftarrow \Sigma.\text{Join}(k_{RA}, \text{gpk}_G, \text{ID}_j, \text{vps}_{\text{ID}_j})$ $\text{CML} \leftarrow \text{ID}_j$ OUTPUT $\text{Ax}_{\text{ID}_j}, \text{gpk}_G', \text{Gl}_G' \text{ID}_j$ ReceivePw(pw): OUTPUT $\Sigma.\text{Verify}(\text{gpk}_G, pw, T_e, \text{RL}_G)$ OpenID(pw, T): $\text{OL} \leftarrow pw$ OUTPUT $\Sigma.\text{IDOpen}(k_{RA}, \text{gpk}_G, pw, T)$ RevokeID(ID_j): OUTPUT $\text{RL}_G' \leftarrow \Sigma.\text{Revoke}(k_{RA}, \text{ID}_j, \text{gpk}_G, \text{RL}_G, \text{Gl}_G)$
---	---

Fig. 2: Procedures Used to Define the Security of a DGTOTP Scheme.

5 New DGTOTP Constructions

In this section, we introduce two new DGTOTP constructions with tailored registration strategies for specific real-world settings, respectively.

Meanwhile, we consider the following four functional goals (FG) in our designs. *FG1) Minimize mount point wastage.* The scheme should reduce unused mount points on the RA side to improve resource utilization and avoid unnecessary computation. *FG2) Support offline verifier.* Verifiers should be able to authenticate members without requiring real-time communication with RA, which is essential for deployment in disconnected or intermittently connected environments. *FG3) Dynamic group management.* The scheme should support member joining and revocation without altering the group public key or the local state of existing members, while preserving the lightweight feature of TOTP. *FG4) Lightweight client-side computation.* The computational burden on each member should be kept low to ensure practicality for resource-constrained devices.

We begin our exploration of efficient DGTOTP constructions by introducing NWDGT, a no-wastage DGTOTP scheme tailored for the delayed-batch registration setting. It eliminates mount point wastage and supports offline verification, but at the cost of increased registration latency. We introduce this scheme first as a natural baseline in our pursuit of reducing wastage in DGTOTP designs. Next, we present LWDGT, a low-wastage DGTOTP construction designed for

the instant-response registration setting. It preserves the security and offline-verification features of NWDGT without registration latency, making it suitable for latency-sensitive environments. Moreover, it achieves lower mount point wastage than the DGTOne scheme. The constructions of NWDGT and LWDGT are summarized in Figure 3, where the part above the dashed line corresponds to the existing construction and the part below represents the additional construction proposed in this work, before we proceed to their detailed descriptions.

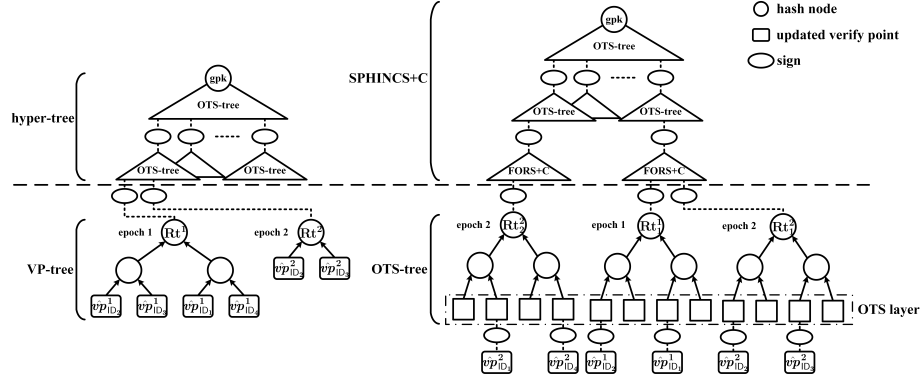


Fig. 3: Constructions of NWDGT (left) and LWDGT (right). In NWDGT, ID_1 and ID_4 leave the group in the 2nd verify epoch; in LWDGT with $U = 8$ and $l = 4$, ID_3 and ID_4 join the group in the 2nd verify epoch.

5.1 NWDGT

We outline the construction challenges and design ideas of NWDGT below. The detailed algorithms and security analysis of NWDGT are given in Appendix B.

Construction Challenges and Ideas. In DGTOne scheme, fixed-size sub-trees are pre-generated per epoch under an assumed maximum group size. Leaves of these sub-trees serve as mount points, signing member-generated verification points. However, in dynamic environments with unpredictable member arrivals, this leads to unused mount points, violating *FG1*. A natural idea is to let sub-tree sizes adapt to actual registrations, but this raises new challenges: dynamic sub-tree generation must not alter group public key or disrupt enrolled members (*FG3*), and it must preserve anonymity, traceability, offline verification (*FG2*), and lightweight computation (*FG4*) in the presence of adaptive adversaries capable of registering honest or malicious members and corrupting honest members.

To address these issues, we propose a no-wastage DGTOTP scheme NWDGT. We implement a register-and-build-later (RBL) strategy by constructing each epoch's sub-tree dynamically based on the precise number of registered verification points to fulfill *FG1*. To fully implement the RBL strategy and facilitate the

dynamic construction of epoch-specific sub-trees under the restriction of *FG3* and *FG4*, we adopt a hash-based hyper-tree structure used in SPHINCS+C [19]. This hyper-tree consists of several layers, where each layer’s leaves are generated using a one-time signature (OTS) scheme. Specifically, the leaves on the lowest layer of this structure are used to sign the root of each sub-tree, and the leaves on higher layers sign the roots of the sub-trees below them.

Instead of signing each verification point individually, RA constructs a verification point tree (VP-tree) using all verification points of an epoch as its leaves and signs the VP-tree root using the hyper-tree. Since the epoch associated with each verification point is public and only verification points from different members within the same epoch need to be randomized, each epoch’s VP-tree root is signed in order using the hyper-tree leaves. To ensure traceability, each verification point is bound to the member’s encrypted identity before constructing the VP-tree, allowing RA to recover identities if necessary. To achieve *FG2*, RA generates and returns all necessary authentication credentials to group members before the beginning of each epoch. These credentials are sufficient for verifiers to authenticate members without needing to download any additional verification information or communicate with RA during the authentication process.

5.2 LWDGT

Construction Challenges and Ideas. While NWDGT eliminates mount point wastage via dynamic VP-tree construction, it incurs undesirable delays, problematic in latency-sensitive settings. Devices registering offline for future epochs must later reconnect solely to obtain credentials, increasing overhead, energy use, and degrading usability. To address this, we propose constructing multiple smaller sub-trees per epoch, enabling RA to process verification points in smaller clusters and thereby improving registration responsiveness while balancing mount point efficiency (*FG1*) and low-latency registration.

Nonetheless, this approach introduces new challenges. If RA still constructs multiple smaller VP-trees, it must wait for enough verification points to form each sub-tree. In sparse or unpredictable arrivals, members may face indefinite registration handling delays. Furthermore, reusing the hyper-tree from NWDGT and mounting sub-trees in a fixed order introduces privacy concerns, as the mounting sequence may reveal members’ relative join times, potentially compromising their anonymity. One potential solution is to mount all sub-trees strictly according to their epoch index, ensuring that sub-trees for earlier epochs appear before those of later epochs. This would eliminate the correlation between registration time. However, this is difficult to enforce in practice, since RA cannot predict the number of members who will register in a given epoch. As a result, future preregistrations may cause later sub-trees to be built earlier. In such a case, to avoid delaying registration, RA must mount available sub-trees immediately, without waiting for sub-trees of earlier epochs. This unpredictability prevents RA from pre-allocating positions for earlier-epoch sub-trees, making it challenging to implement a mounting strategy that preserves privacy under dynamic registration.

To address these limitations, we propose a low-wastage DGTOTP scheme LWDGT based on one-time signature trees (OTS-trees). We implement a partitioned-build-on-demand (PBOD) strategy. The key idea is to allow RA to instantiate a small OTS-tree only when a registration event occurs, instead of constructing VP-trees that require a complete batch of verification points. This on-demand generation not only enables immediate processing of incoming registration requests but also avoids creating unnecessary mount points, thus fulfilling *FG1*. In this design, each verification point is signed and mounted by a leaf of a newly or existing OTS-tree associated with the corresponding epoch.

To preserve anonymity, a keyed permutation is applied to randomize member placement across OTS-trees. Specifically, each epoch is partitioned into U/l OTS-trees, where U is the maximum group size and l is the OTS-tree size. RA assigns each member a unique identifier and applies a keyed permutation uniquely for each epoch to shuffle the identifiers. The shuffled set is then divided into clusters of size l , determining randomized member positions across OTS-trees. This conceals registration timing and prevents inference of join order. To deal with potential collisions arising from random OTS-trees placement, we adopt the full SPHINCS+C. This ensures secure mounting and supports concurrent mounting of multiple OTS-trees while satisfying *FG4* under the restriction of *FG3*. As in NWDGT, traceability is achieved by binding each verification point to the encrypted identity before it is signed, and offline verification (*FG2*) is enabled because RA returns all authentication credentials to members at registration.

Detailed Construction. Both the digital signature SPHINCS+C and the one-time signature OTS used below share the syntax of digital signature. Below are the algorithms' details of our construction (pseudocodes of Join and Verify algorithms are given in Appendix C).

- **Setup**($1^\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e$): RA samples a secret key k_{RA} , sets T_s to the current time T_c , and computes $E := \frac{(T_e - T_s)}{\Delta_{vp}}$ and $N := \frac{(T_e - T_s)}{\Delta_{pw}}$, where E denotes the number of TOTP instances and N denotes the number of passwords per instance. Next, RA initializes the building blocks including a pseudo-random function F as $\text{pms}_{\text{PRF}} := F.\text{Setup}(1^\kappa)$, an authenticated symmetric encryption ASE as $\text{pms}_{\text{ASE}} := \text{ASE}.\text{Setup}(1^\kappa)$, a collision-resistant hash function H as $\text{pms}_{\text{CRH}} := H.\text{Setup}(1^\kappa)$, and a permutation PM as $\text{pms}_{\text{PM}} := \text{PM}.\text{Setup}(1^\kappa)$. RA then generates the SPHINCS+C public key $pk_{\text{SPHINCS+C}}$ by computing $(\text{pms}_{\text{SPHINCS+C}}, pk_{\text{SPHINCS+C}}, sk_{\text{SPHINCS+C}}) \leftarrow \text{SPHINCS+C}.\text{Setup}(1^\kappa)$, with the SPHINCS+C secret key $sk_{\text{SPHINCS+C}}$ also generated, and sets the group public key $\text{gpk}_G := pk_{\text{SPHINCS+C}}$. Moreover, for each epoch, RA carries out the following preparation steps: (i) compute the i -th permutation key $k_{\text{PM}}^i := F(k_{RA}, \text{"PM"} \| i)$ and the permuted set $S_i = \{s_1^i, \dots, s_U^i\} := \text{PM}.\text{Shuffle}(k_{\text{PM}}^i, \{1, \dots, U\})$; (ii) partition set S_i into $\frac{U}{l}$ clusters of size l , denoted by $Cl_i = \{\{s_1^i, \dots, s_l^i\}, \{s_{l+1}^i, \dots, s_{2l}^i\}, \dots\}$.

Meanwhile, RA maintains the group management information Gl_G , which includes a registration list IDL_G for storing member identities and a set of variables $\{\text{RS}_{s_j^i} := (pk_{s_j^i}^i, C_{s_j^i}^i)\}_{j \in [U]}$ for currently active verify epochs. These

variables are periodically updated by RA to remove expired entries and prepare for the next epoch. RA also manages a revocation list RL_G , a bit array of length U , where each bit (initialized to 0) indicates whether the corresponding member is revoked.

- $\text{MInit}(\text{ID}_j)$: Let τ denote the consecutive epochs a member must register during the current registration period. τ is pre-defined by the system through a fixed division of the system's lifetime, representing epochs required for registration at this time (e.g., epochs 1-10).⁶ A group member ID_j first runs $(\text{pms}_{\text{PRF}}, k_{\text{ID}_j}) \leftarrow \text{Setup}(1^\kappa)$ to sample a random key $k_{\text{ID}_j} \xleftarrow{\$} \mathcal{K}_{\text{PRF}}$ as its secret key. For each epoch $i \in \tau$, ID_j runs $\text{TOTP.Setup}(1^\kappa, T_i, T_i + \Delta_{vp}, \Delta_{pw})$ to initialize i -th TOTP instance, where $T_i = T_{i-1} + \Delta_{vp}$ and $T_0 = T_s$. Next, it computes i -th verification point $vp_{\text{ID}_j}^i := \text{TOTP.PInit}(sd_{\text{ID}_j}^i)$, where $sd_{\text{ID}_j}^i := F(k_{\text{ID}_j}, \text{ID}_j \| i)$ is the corresponding secret seed. This algorithm returns k_{ID_j} and $\text{vps}_{\text{ID}_j} = \{vp_{\text{ID}_j}^i\}_{i \in \tau}$.
- $\text{Join}(k_{\text{RA}}, \text{gpk}_G, \text{ID}_j, \text{vps}_{\text{ID}_j})$: We assume that all identities recorded in IDL_G are ordered by their joining time. Namely, RA maps each member ID_j to a number $\alpha_{\text{ID}_j} \in [U]$ based on its join order. Once the position of a member in IDL_G is fixed, it remains unchanged throughout the lifespan of the group. For $i \in [E]$, RA handles registrations of all parties. Specifically, if a party ID_j submits registration of the i -th epoch, RA generates and returns the corresponding auxiliary information $\text{Ax}_{\text{ID}_j} = (\sigma_{\text{ID}_j}^i, C_{\text{ID}_j}^i)$, where $\sigma_{\text{ID}_j}^i := (\sigma_{\text{OTS}}^i, \text{Pf}_{\text{ID}_j}^i, \sigma_{\text{SPHINCS+C}}^i)$. On receiving a party's registration request, RA does the following steps: (i) compute the i -th ASE key $k_{\text{ASE}}^i := F(k_{\text{RA}}, \text{"ASE"} \| i)$, the identity ciphertext $C_{\text{ID}_j}^i := \text{ASE.Enc}(k_{\text{ASE}}^i, \text{ID}_j)$ (note that ASE.Enc is randomized, so $C_{\text{ID}_j}^i \neq C_{\text{ID}_j}^{i+1}$), and the updated verification point $\hat{vp}_{\text{ID}_j}^i := H(vp_{\text{ID}_j}^i \| C_{\text{ID}_j}^i)$; (ii) find the index idx such that $s_{idx}^i = \alpha_{\text{ID}_j}$, and generate l OTS key pairs $\{(pk_j^i, sk_j^i)\}$ for indices $j \in [l(\lceil \frac{idx}{l} \rceil - 1) + 1, \dots, l\lceil \frac{idx}{l} \rceil]$ by initializing l independent OTS instances using $\text{OTS.Setup}(1^\kappa)$; (iii) compute the OTS-tree $\text{MT}_{\lceil \frac{idx}{l} \rceil}^i := \text{MT.Build}(P_{\lceil \frac{idx}{l} \rceil}^i)$, where $P_{\lceil \frac{idx}{l} \rceil}^i := \{pk_j^i\}_{j \in \{l(\lceil \frac{idx}{l} \rceil - 1) + 1, \dots, l\lceil \frac{idx}{l} \rceil\}}$; (iv) compute OTS signature $\sigma_{\text{OTS}}^i := \text{Sign}(sk_{idx}^i, \hat{vp}_{\text{ID}_j}^i)$ and the Merkle proof as $\text{Pf}_{\text{ID}_j}^i := \text{MT.GetPf}(\text{MT}_{\lceil \frac{idx}{l} \rceil}^i, pk_{idx}^i)$; (v) compute SPHINCS+C signature $\sigma_{\text{SPHINCS+C}}^i := \text{SPHINCS+C.Sign}(sk_{\text{SPHINCS+C}}, \text{MT}_{\lceil \frac{idx}{l} \rceil}^i)$.
- $\text{GetSD}(k_{\text{ID}_j}, T)$: This algorithm returns i -th secret seed $sd_{\text{ID}_j}^i := F(k_{\text{ID}_j}, \text{ID}_j \| i)$, where $i := \frac{T - T_s}{\Delta_{vp}}$ is the index of the TOTP instance.
- $\text{PwdGen}(k_{\text{ID}_j}, T)$: The party ID_j first computes the index as $i := \frac{T - T_s}{\Delta_{vp}}$. Next, it computes the secret seed $sd_{\text{ID}_j}^i := F(k_{\text{ID}_j}, \text{ID}_j \| i)$ and the password $\bar{pw}_{\text{ID}_j}^{i,z} := \text{TOTP.PGen}(sd_{\text{ID}_j}^i, T_c)$, where $z := \frac{T_c - T_s - i \cdot \Delta_{vp}}{\Delta_{pw}}$ is the index of password in the i -th epoch. Then, the password $pw_{\text{ID}_j}^{i,z} := (\bar{pw}_{\text{ID}_j}^{i,z}, \sigma_{\text{ID}_j}^i, C_{\text{ID}_j}^i)$ is returned.

⁶ Such design supports manual updates of group management information to verifiers at epoch boundaries, allowing them to operate offline.

- **Verify**($\text{gpk}_G, pw_{ID_j}^{i,z}, T, \text{RL}_G$): The verifier first checks whether the member is revoked by consulting the local registration status list $\{\text{RS}_{s_j^i}\}_{j \in [U]}$ maintained and published by RA. If valid, it verifies the correctness of the TOTP password and reconstructs the verification point for signature validation. Using the received Merkle proof $\text{Pf}_{ID_j}^i$ from member and the stored OTS-tree public key from registration status, it derives the corresponding OTS-tree root, which is then used to verify the SPHINCS+C signature for the epoch. Specifically, the verifier proceeds as follows: (i) return 0, if $C_{ID_j}^i$ is not contained in $\{\text{RS}_{s_j^i}\}_{j \in [U]}$; (ii) get the position idx such that RS_{idx} contains the identity ciphertext $C_{ID_j}^i$; (iii) compute the verification point $vp_{ID_j}^i$ from $\bar{p}w_{ID_j}^{i,z}$ and T , and $\bar{v}p_{ID_j}^i := H(vp_{ID_j}^i \| C_{ID_j}^i \| i)$; (iv) compute the tree root Rt based on pk_{idx}^i and $\text{Pf}_{ID_j}^i$; (v) output 1, if and only if $\text{TOTP.Verify}(vp_{ID_j}^i, \bar{p}w_{ID_j}^{i,z}, T) = 1$, $\text{OTS.Verify}(pk_{idx}^i, \bar{v}p_{ID_j}^i, \sigma_{\text{OTS}}^i) = 1$, and $\text{SPHINCS+C.Verify}(\text{gpk}_G, Rt, \sigma_{\text{SPHINCS+C}}^i) = 1$.
- **Revoke**($k_{RA}, ID_j, \text{gpk}_G, \text{RL}_G, \text{Gl}_G$): Let $\alpha_{ID_j} := \text{IDL}_G[ID_j]$ denote the mapped number that corresponds to ID_j . To revoke a group member ID_j , RA sets $\text{RL}_G[\alpha_{ID_j}] := 1$, and stops updating RS relevant to ID_j in all future epochs.
- **IDOpen**($k_{RA}, \text{gpk}_G, pw_{ID_j}^{i,z}, T$): RA aborts if $\text{Verify}(\text{gpk}_G, pw_{ID_j}^{i,z}, T, \text{RL}_G) = 0$. Otherwise, RA returns $ID_j := \text{ASE.Dec}(k_{ASE}^i, C_{ID_j}^i)$, where $k_{ASE}^i := F(k_{RA}, \text{"ASE"} \| i)$.

Security Analysis. Our construction achieves both anonymity and traceability, as stated by the two theorems below.

Theorem 1. *Suppose that the building blocks TOTP, H, PRF, SPHINCS+C, OTS, ASE, MT and PM are secure, then LWDGT provides anonymity.*

The proof for Theorem 1 is in Appendix D. Here, we informally analyze how our scheme achieves anonymity. The prover authenticates by sending a password comprising four components: a TOTP password, an identity ciphertext, an OTS-tree signature, and a SPHINCS+C signature. The hash chain-based TOTP instances leverage a pseudo-random function to compute the secret seed, and compute the TOTP password and the verification point by a collision-resistant hash function. The output of the hash function and the ASE encryption algorithm contain randomness; thus, an attacker cannot distinguish the updated verification points of the honest members. Because of the employment of an unpredictable permutation function to determine the position of all updated verification points in the OTS-tree, an attacker cannot associate the SPHINCS+C signature and the OTS-tree signature with the updated verification points of the honest members and therefore cannot distinguish the correspondence between the verification points indicated by the leaf nodes of the OTS-tree mounted by SPHINCS+C and the honest members. Finally, since identity is encrypted by an ASE encryption algorithm, the attacker learns nothing about the identity from identity ciphertext.

Theorem 2. *Under the same assumptions as in Theorem 1, LWDGT provides traceability.*

The proof of Theorem 2 is in Appendix E. Here, we provide an informal analysis to convey the intuition behind the traceability property. Attackers cannot forge a valid password that can be traced to an honest group member. Due to the collision-resistant hash function, attackers are unable to forge a valid TOTP password. As a result of the employment of IND-CCA ASE encryption to generate group member identity ciphertext, attackers are unable to distinguish between group members; thus, they are unable to forge a password that binds to the identity of an honest group member. In addition, a secure OTS-tree scheme and a SPHINCS+C scheme make sure that attackers are unable to forge signatures of an OTS-tree and SPHINCS+C. On the other hand, RA can use its secret key to decrypt the identity ciphertext with ASE to trace the identity of the group member corresponding to the password.

6 Evaluation

Experiment Setup. We implement NWDGT and LWDGT ⁷ using the OpenSSL library and conduct experiments on a PC (AMD Ryzen 9 7945HX, 16 GB RAM) acting as RA and verifier, and a Raspberry Pi 3 emulating resource-constrained clients in mobile access scenarios, such as smartcards, IoT tokens, or embedded user devices. The TOTP scheme is instantiated with T/Key [20], SHA256 is used as the hash function, 128-bit AES-GCM-SIV [16] for authenticated symmetric encryption, and Fisher-Yates shuffle algorithm [13] for the permutation scheme.

In NWDGT, the HT follows the SPHINCS+C hyper-tree structure [19]. In LWDGT, SPHINCS+C is instantiated as full SPHINCS+C, with its WOTS+C for one-time signature scheme OTS. Based on our following theoretical analysis of mount point wastage, we set the size of each OTS-tree in LWDGT to $l = 2$ to minimize the expected waste rate during randomized sub-tree mounting.

We adopt the small-signature parameter set from the SPHINCS+C specification to ensure 128-bit security, minimizing client storage at the cost of slightly higher verification time. For both schemes, we set the total height of the hyper-tree to $h = 66$, the number of layers to $d = 11$, and the Winternitz parameter to $w = 128$. Following [10], the password interval is set to $\Delta_{pw} = 5$ seconds and the verification point lifespan to $\Delta_{vp} = 5$ minutes, yielding a password chain length $N = 60$. For LWDGT, which includes the FORS+C layer, we additionally configure the number of FORS trees as $k = 9$, the number of leaves per tree as $t = 2^{13}$, and the number of leaves in the optional extra FORS tree as $t' = 2^{18}$.

Theoretical Analysis of Wastage Rate. We analyze mount point efficiency using both analytical bounds and probabilistic modeling.

Theoretical Comparison. We define the overall wastage rate as $\text{Waste} = 1 - \frac{\sum_{i=1}^E x_i}{C}$, where x_i denotes the number of registrations in epoch i , E is the total number of epochs, and C is the total capacity provisioned across all epochs. This measures the fraction of unused mount points across the system lifespan. NWDGT incurs no wastage due to its adaptive registration strategy, while LWDGT

⁷ Code is available at: <https://github.com/LannisterArthur/Efficient-DGTOTP>

may incur some wastage to support instant-response registration, but its wastage remains strictly bounded. Lemma 1 shows that, under equal registration counts across all epochs, the wastage rate under LWDGT never exceeds that of DGTOne.

Lemma 1. *Let U , E , and $l \in [2, U]$ be fixed. If LWDGT and DGTOne register same number of verification points per epoch, then $\text{Waste}_{\text{LWDGT}} \leq \text{Waste}_{\text{DGTOne}}$.*

Proof. The overall wastage rate is defined as $\text{Waste} = 1 - \frac{\sum_{i=1}^E x_i}{C}$, where C denotes the total provisioned capacity across all epochs. Since both schemes register the same total number of points $\sum_{i=1}^E x_i$, the comparison reduces to their provisioned capacities C_{DGTOne} and C_{LWDGT} . DGTOne allocates U mount points per epoch, giving $C_{\text{DGTOne}} = U \cdot E$. In LWDGT, each epoch constructs up to $\frac{U}{l}$ sub-trees of size l , yielding $C_{\text{LWDGT}} \leq U \cdot E = C_{\text{DGTOne}}$. Substituting these into the wastage rate definition gives

$$\text{Waste}_{\text{LWDGT}} = 1 - \frac{\sum_{i=1}^E x_i}{C_{\text{LWDGT}}} \leq 1 - \frac{\sum_{i=1}^E x_i}{C_{\text{DGTOne}}} = \text{Waste}_{\text{DGTOne}}.$$

That proves the lemma. $\square$

While this bound guarantees that LWDGT never performs worse than DGTOne, it is important to analyze the probability of equality because achieving the same wastage rate implies that the same number of mount points remain unused as in DGTOne. This directly limits the number of members the verifier can authenticate offline and reduces overall mount point utilization, which is undesirable in mobile and offline scenarios.

For this purpose, we define event A_i as the case in epoch i where LWDGT provisions the same capacity as DGTOne under the same registrations x_i , i.e., $C_{\text{LWDGT}}^{(i)} = C_{\text{DGTOne}}^{(i)} = U$. When $0 \leq x_i < \frac{U}{l}$, then LWDGT provisions only x_i sub-trees of size l , resulting in $C_{\text{LWDGT}}^{(i)} < U = C_{\text{DGTOne}}^{(i)}$, and hence $\text{Waste}_{\text{LWDGT}}^{(i)} < \text{Waste}_{\text{DGTOne}}^{(i)}$ in that epoch. When $\frac{U}{l} \leq x_i \leq U$, the probability that LWDGT provisions the full capacity $C_{\text{LWDGT}}^{(i)} = U$ is

$$\mathbb{P}[A_i] = \frac{\sum_{j=0}^{U/l} (-1)^j \binom{U/l}{j} \frac{(U-jl)!}{(U-jl-x_i)!}}{\frac{U!}{(U-x_i)!}}.$$

Here, the denominator counts all possible assignments of the x_i registrations to the $\frac{U}{l}$ sub-trees (no restriction), while the numerator counts only those assignments in which every sub-tree receives at least one registration, obtained via the inclusion-exclusion principle.

Therefore, equality $\text{Waste}_{\text{LWDGT}} = \text{Waste}_{\text{DGTOne}}$ occurs only if $x_i \in [\frac{U}{l}, U]$ for every epoch i , and LWDGT provisions full capacity in all such epochs. The overall probability is

$$\Pr[\text{Waste}_{\text{LWDGT}} = \text{Waste}_{\text{DGTOne}}] = \prod_{i=1}^E \Pr[A_i] = \prod_{i=1}^E \begin{cases} 0, & 0 \leq x_i < \frac{U}{l}, \\ \mathbb{P}[A_i], & \frac{U}{l} \leq x_i \leq U, \end{cases}$$

which is negligible for realistic parameters (e.g., $E = 103,680$ in a year), showing that LWDGT almost always achieves strictly better mount point utilization in practice.

Expected Waste under Probabilistic Modeling. To evaluate expected wastage under realistic deployment conditions, we choose parameters that reflect typical small-scale institutional settings. According to [8,26,32], a small company typically has at most 500 employees, which motivates setting the system maximum size $U = 500$. The average monthly number of new registrations $\beta \in \{5, 10, 15, 20\}$ represents typical hiring rates, how the organization grows over time. We model the number of registrations x_i per epoch as a Poisson random variable because it naturally describes count data for independent, sporadic events within a fixed interval [27]. Empirical studies show that small firms hire only a few employees per month on average [32], supporting the use of this model to represent employee growth.

We then construct a probabilistic model over $E = 12y$ epochs (one year with $y = 8640$ epochs per month). The number of registrations x_i follows a Poisson distribution with mean $\lambda_i = \alpha + i \cdot (\beta/y)$, where $\alpha \in [1, U]$ denotes the initial number of registered members. As a result, the probability of observing x_i registrations in epoch i is given by $P_i(x_i) = \frac{(\lambda_i)^{x_i} e^{-\lambda_i}}{x_i!}$. Expected capacity and wastage are computed by enumerating $x_i \in [\alpha, U]$ and weighting by $P_i(x_i)$.

In LWDGT, U members are divided into U/l sub-trees of size l . For registrations x_i , the expected number of used sub-trees is $\mathbb{E}[\text{Used Trees} \mid x_i] = \frac{U}{l} \cdot \left[1 - \binom{U-l}{x_i} / \binom{U}{x_i}\right]$. Thus corresponding capacity and waste are $C_i(x_i) = l \cdot \mathbb{E}[\text{Used Trees} \mid x_i]$ and $W_i(x_i) = C_i(x_i) - x_i$. Aggregating over x_i , we obtain the epoch-level expectations: $\mathbb{E}[W_i] = \sum_{x_i=\alpha}^U P_i(x_i) \cdot W_i(x_i)$, $\mathbb{E}[C_i] = \sum_{x_i=\alpha}^U P_i(x_i) \cdot C_i(x_i)$. Summing over all epochs, the total expected wastage ratio is $\mathbb{E}[\text{Waste}_{\text{LWDGT}}] = \frac{\sum_{i=1}^E \mathbb{E}[W_i]}{\sum_{i=1}^E \mathbb{E}[C_i]}$.

For comparison, we evaluate the expected wastage rate under the DGTOne scheme, where each epoch utilizes a single Merkle tree of fixed capacity U . Then, the total expected wastage rate for DGTOne is $\mathbb{E}[\text{Waste}_{\text{DGTOne}}] = \frac{1}{E} \sum_{i=1}^E \left(1 - \frac{\mathbb{E}[x_i]}{U}\right) = 1 - \frac{1}{U \cdot E} \sum_{i=1}^E \mathbb{E}[x_i]$.

In Figure 4, we compare the expected mount point wastage rates of LWDGT and DGTOne under different values of monthly registrations β and initial registered users α over one year. The results show that as α increases, the expected wastage rates of both schemes exhibit a decreasing tendency, and this trend persists across different β values, indicating that variations in monthly registration do not fundamentally alter the dependence of wastage rate on α . Moreover, the OTS-tree size l also affects the curves: larger l causes LWDGT to gradually approach the wastage of DGTOne, while smaller l leads to more pronounced differences. It is worth noting that even if some users revoke their registrations, the corresponding mount points remain occupied, so revocation does not affect the mount point wastage rate.

This difference in performance becomes particularly notable when the initial number of users is far below the system’s maximum capacity ($\alpha \leq U/2$), a situation often observed in real-world small organizations where the workforce starts small and grows gradually [32]. As illustrated in Figure 4, under these conditions, LWDGT can reduce the annual wastage rate by 12% to 41% when the monthly registration is lower ($\beta = 1\% \cdot U = 5$), and by 6% to 31% when the monthly registration is relatively large ($\beta = 4\% \cdot U = 20$).

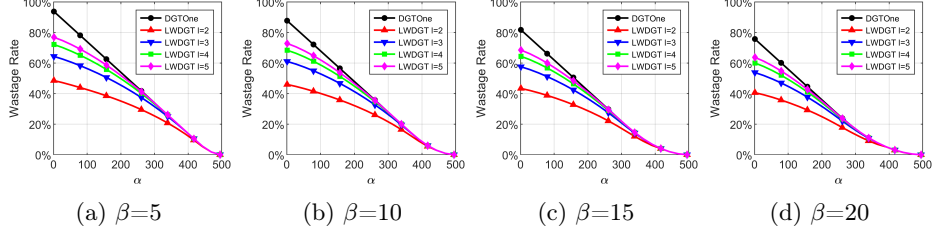


Fig. 4: Comparison of expected wastage rate.

Runtime Performance. We evaluate the computational efficiency of our schemes by measuring the runtime of their algorithms.

RA Setup time. The runtime of **Setup** is primarily determined by constructing the top-level tree of the hyper-tree, which involves generating the leaves and computing the root from these leaves. The time to generate each leaf depends on the Winternitz parameter ω of WOTS+C [19], which specifies the number of bits encoded per step in each WOTS+C hash chain. Larger ω values produce shorter signatures but require more hash evaluations per leaf, while smaller ω values reduce the number of hashes at the cost of longer signatures. The time to compute the top-level root depends on the tree height h/d , where h is the total height of the hyper-tree and d is the number of layers into which the hyper-tree is divided. Figure 5a shows that runtime increases exponentially with h/d , and that larger ω substantially increases runtime due to more hash computations per leaf. Because the hyper-tree is divided into layers, h/d remains small (typically less than 10), **Setup** is still practical.

Member Initialization Time.

Mlinit in both schemes runs one Eval of PRF and N hash evaluations to initialize a TOTP instance in each epoch. In our experiments, we assume that a member registers for all E epochs, and Figure 5b shows the computational cost measured on a Raspberry Pi for different TOTP chain lengths $N = 30$ and $N = 60$, representing shorter and longer TOTP chains, respectively. From the figure, it can be observed that the total runtime increases linearly with the number of epochs E . This linear growth

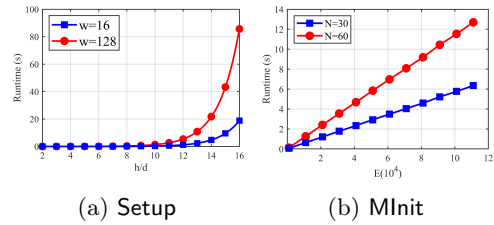


Fig. 5: Runtimes of Setup and Mlinit.

occurs because each epoch’s initialization requires a fixed amount of computation. Nonetheless, the measured per-epoch initialization time is approximately $0.07ms$ for $N = 30$ and $0.12ms$ for $N = 60$, demonstrating that Minit is still practical even for longer chains.

Join Time. In both schemes, the Join algorithm begins by encrypting the member’s identity using ASE, requiring one ASE encryption with runtime $0.57 \mu s$. The subsequent credential generation differs between the two schemes. In NWDGT, RA aggregates registrations over an epoch and constructs a single VP-tree for all members. Once all registrations are collected, constructing the VP-tree and preparing credentials requires $U - 1$ hash evaluations in the worst case or $U/2 - 1$ in the average case (for Merkle proof generation), along with a single hyper-tree signing operation to produce the associated signature. For $U = 2^{20}$, the total join cost for an epoch reaches $23.01s$ in the worst case and $12.29s$ in the average case. In LWDGT, credentials are generated immediately upon registration, requiring one hash evaluation for the updated verification point, one OTS-tree signing (with $l = 2$), and one SPHINCS+C signing computation, with per-join runtime approximately $1.07s$.

Password Generation Time. Since RA handles the most expensive computations (i.e., credentials generation), members only need to compute TOTP passwords: one hash function in the average case, and one more F in the worst case to switch TOTP instances. The runtime is $1.54 \mu s$ (average) and $23.64 \mu s$ (worst).

Group Management Information Update Time. In both schemes, RA updates the registration status $RS_{s_j^i}$ in part of the join process. Since credential generation already produces the necessary verification materials, we assume RA stores these materials at registration time. Thus, updating group management information imposes no additional runtime overhead during verification and does not require separate update procedures. We further assume that each member submits verification points for a predefined time window (e.g., a month or a year) at registration. This allows RA to perform batch updates manually and periodically distribute group management information to verifiers. Such a design ensures that verifiers can operate offline without access to RA.

Password Verification Time. In NWDGT, verification begins with checking the TOTP password, requiring up to $N = 60$ hash evaluations in the worst case and $N/2$ on average. The verifier then computes one hash to derive the VP-tree leaf, followed by $\log U$ hash evaluations in the worst case or $\log(U/2)$ on average to compute the VP-tree root. Verification concludes with a hyper-tree signature check, which dominates the runtime. The difference between worst and average cases amounts to only 31 extra hash evaluations, corresponding to $2.56 \mu s$, making the verification time effectively stable. The measured total verification time is approximately $7.54 ms$. In LWDGT, the verifier similarly begins with TOTP password validation, followed by one hash computation for the updated verification point and one OTS signature verification algorithm to validate this value using the accompanying OTS signature. It then performs $\log l$ hash evaluations to compute the OTS-tree root, and finally runs one SPHINCS+C

verification algorithm to verify the correctness of the SPHINCS+C signature on that root. The total runtime is approximately 8.05 *ms*.

Identity Open Time. The overhead of IDOpen in both schemes consists of the costs of the respective verification algorithm cost and one decryption of ASE. The runtime of one ASE.Dec is 0.55 μ s.

Comparison. We compare our schemes with YJN+ [34] and DGTOne [10] schemes in terms of several properties. Table 1 summarizes the comparison across four dimensions. These include whether it supports a dynamic group setting, whether it avoids mount point wastage, whether the verifier can operate without online connectivity, and how the registration process is handled by RA.

Table 1: Comparison of Properties.

Schemes	Group Setting	No Wastage	Offline Verifier Support	Registration Handling
YJN+ [34]	Static	Yes	Yes	Immediate
DGTOne [10]	Dynamic	No	No	Immediate
NWDGT	Dynamic	Yes	Yes	Delayed
LWDGT	Dynamic	No	Yes	Immediate

Table 2 compares the performance of these schemes under worst-case (Wor) and average-case (Avg) settings. In evaluating setup costs, we compare the GVSTGen algorithm of YJN+, RASetup of DGTOne, and the Setup used by our two schemes, as all serve the purpose of group public key generation. We use ‘BFc’ and ‘BFi’ to represent the costs of the check and insert operations of the Bloom filter, respectively. ‘Ae’ and ‘Ad’ denote AES-GCM-SIV encryption and decryption algorithms, while ‘Ss’ and ‘Se’ denote AES setup and encryption algorithms. ‘CHe’, ‘CHs’, and ‘CHc’ correspond to the evaluation, setup, and collision-finding algorithms of the chameleon hash function. We denote the cost of permutation with ‘PM’. Additionally, we use ‘Wp’, ‘Ws’, and ‘Wv’ to represent the public key generation, signing, and verification algorithms of WOTS+C, and ‘Fs’ and ‘Fv’ to denote the signing and verification algorithms of FORS+C. We use ‘ μ ’ and ‘ ν ’ denote ‘ $E^{\frac{1}{d}}$ ’ and ‘ $2^{h/d}$ ’.

From the two tables, it is evident that both of our proposed schemes support dynamic group management (not provided in YJN+ scheme) and offline verifier functionality (not provided in DGTOne scheme) while maintaining low overhead. In particular, group public key generation in our schemes leverages a hyper-tree structure, where only a small tree at the top layer needs to be constructed, greatly reducing setup complexity. Password generation is lightweight, requiring only the generation of a TOTP password. While member-side storage in our schemes is slightly higher than DGTOne due to offline verifier support, expensive credential generation is handled by RA, keeping the schemes suitable for resource-constrained clients. Importantly, mount points are efficiently utilized: NWDGT achieves zero wastage, and LWDGT significantly reduces wastage compared to DGTOne, demonstrating practicality and scalability of our designs.

Acknowledgments. This work was supported by the National Key Research and Development Program of China under Grant No. 2025YFE0220300 and the National Natural Science Foundation of China under Grant No. 62372386.

Table 2: Comparison of performance.

Schemes		Execution Time							Storage Cost (Byte)			Password size(Byte)		
		RASetup	MInit	Revoke	Join	GetSD	PwdGen	GMUpdate	Verify	Open	Member		Verifier	RA
YJN+	Avg	$2U \cdot E \cdot H + \phi \cdot BF_i$	$60E \cdot H + 2E \cdot Se$	-	-	2Sc	30H	-	$(32 + \log^{\frac{U}{2}})H + 1BF_c$	$(32 + \log^{\frac{U}{2}})H + 1BF_c + 1Ad$	$1004E \cdot \log^{\frac{U}{2}}$ +16	$1.44e \cdot \phi$	16	$76 + 32 \log^{\frac{U}{2}}$
	Wor	$2U \cdot E \cdot H + \phi \cdot BF_i$	$60E \cdot H + 2E \cdot Se$	-	-	2Sc	$60H + 2Se$	-	$(62 + \log^{\frac{U}{2}})H + 1BF_c$	$(62 + \log^{\frac{U}{2}})H + 1BF_c + 1Ad$				
DGTOne	Avg	$6U \cdot E \cdot Ae + (U + 1)Ae + E \cdot PM + U \cdot E \cdot CH_e + U \cdot E \cdot CH_s + (U \cdot E - 1)H$	1Ss	1Ass	1Se	2Se	30H	$(9U + 1)Se + 1PM + U \cdot CH_e + U \cdot Ae + U \cdot CH_s$	$(32 + \log^{\frac{U}{2}})H + 1CH_e$	$(32 + \log^{\frac{U}{2}})H + 1CH_e + 3Se + 1PM + 1Ad$	36	$32 + 460U + 32 \log^E$	$16 + 5U + 32E + 32E \cdot \log^E$	108
	Wor	$6U \cdot E \cdot Ae + (U + 1)Ae + E \cdot PM + U \cdot E \cdot CH_e + U \cdot E \cdot CH_s + (U \cdot E - 1)H$	1Ss	1Ass	1Se	2Se	$62H + 1CH_e + 10Se + 1Ae$	$(9U + 1)Se + 1PM + U \cdot CH_e + U \cdot Ae + U \cdot CH_s$	$(62 + \log^{\frac{U}{2}})H + 1CH_e$	$(62 + \log^{\frac{U}{2}})H + 1CH_e + 3Se + 1PM + 1Ad$				
NWDGT	Avg	$\mu(Wp + H)$	$30E \cdot H + 1Se \cdot E$	1Ass	$d\mu(Wp + H) + d \cdot Ws + 1PM + 1Ae + U/2 \cdot H$	2Se	30H	Performed in Join	$(30 + \log^U)H + d \cdot Wv$	$(30 + \log^U)H + d \cdot Wv + 1Ad$	$60 + 4d + 16 \log^E + 2048d / \log^w$	$32 + 44U$	16	$76 + 4d + 16 \log^E + 2048d / \log^w$
	Wor	$\mu(Wp + H)$	$60E \cdot H + 2E \cdot Se$	1Ass	$d\mu(Wp + H) + d \cdot Ws + 1PM + 1Ae + U/2 \cdot H$	2Se	$60H + 2Se$	Performed in Join	$(61 + \log^U)H + d \cdot Wv$	$(61 + \log^U)H + d \cdot Wv + 1Ad$				
LWDGT	Avg	$\nu(Wp + H) + E \cdot PM$	$30E \cdot H + 1E \cdot Se$	1Ass	$(dv + 1)Wp + (l + dv)H + 1Fs + 1Ae + d \cdot Ws$	2Se	30H	Performed in Join	$(30 + l)H + 1Fv + (d + 1)Wv$	$(30 + l)H + (d + 1)Wv + 1Fv + 1Ad$	$72 + 4d + 16h + 16k(\log^l + 1) + 4096d / \log^w$	$32 + 60U$	16	$88 + 4d + 16 \log^l + 16(h + kl) + 16k \log^l + 4096d / \log^w$
	Wor	$\nu(Wp + H) + E \cdot PM$	$60E \cdot H + 2Se \cdot E$	1Ass	$(dv + 1)Wp + (l + dv)H + 1Fs + 1Ae + d \cdot Ws$	2Se	$60H + 2Se$	Performed in Join	$(60 + l)H + 1Fv + (d + 1)Wv$	$(60 + l)H + (d + 1)Wv + 1Fv + 1Ad$				

References

- Backes, M., Hanzlik, L., Schneider-Bensch, J.: Membership privacy for fully dynamic group signatures. In: CCS. pp. 2181–2198. ACM (2019)
- Bellare, M., Micciancio, D., Warinschi, B.: Foundations of group signatures: Formal definitions, simplified requirements, and a construction based on general assumptions. In: EUROCRYPT. pp. 614–629. Springer (2003)
- Bellare, M., Rogaway, P.: The security of triple encryption and a framework for code-based game-playing proofs. In: EUROCRYPT. pp. 409–426. Springer (2006)
- Bellare, M., Shi, H., Zhang, C.: Foundations of group signatures: The case of dynamic groups. In: CT-RSA. pp. 136–153. Springer (2005)
- Boneh, D., Boyen, X., Shacham, H.: Short group signatures. In: CRYPTO. pp. 41–55. Springer (2004)
- Boneh, D., Shacham, H.: Group signatures with verifier-local revocation. In: CCS. pp. 168–177. ACM (2004)
- Bootle, J., Cerulli, A., Chaidos, P., Ghadafi, E., Groth, J.: Foundations of fully dynamic group signatures. J. Cryptol. **33**(4), 1822–1870 (2020)
- Bort, J., Wiklund, J., Yu, W.: Firm growth and the job satisfaction of the startup workforce. Strategic Management Journal (2025)
- Camenisch, J., Drijvers, M., Hajny, J.: Scalable revocation scheme for anonymous credentials based on n-times unlinkable proofs. In: WPES@CCS. pp. 123–133. ACM (2016)
- Cao, X., Yang, Z., Ning, J., Jin, C., Lu, R., Liu, Z., Zhou, J.: Dynamic group time-based one-time passwords. IEEE Trans. Inf. Forensics Secur. **19**, 4897–4913 (2024)
- Chaum, D., van Heyst, E.: Group signatures. In: EUROCRYPT. pp. 257–265. Springer (1991)
- Chen, L., Dong, C., Newton, C.J.P., Wang, Y.: Sphinx-in-the-head: Group signatures from symmetric primitives. ACM Trans. Priv. Secur. **27**(1), 11:1–11:35 (2024)
- Fisher, R.A., Yates, F.: Statistical tables for biological, agricultural and medical research, pp. 26–27. Oliver & Boyd, Third edn. (1948)

14. Garms, L., Lehmann, A.: Group signatures with selective linkability. In: Public Key Cryptography (1). Lecture Notes in Computer Science, vol. 11442, pp. 190–220. Springer (2019)
15. Groth, J.: Fully anonymous group signatures without random oracles. In: ASIACRYPT. pp. 164–180. Springer (2007)
16. Gueron, S.: Aes-gcm-siv implementations (128 and 256 bit) (2018), <https://github.com/Shay-Gueron/AES-GCM-SIV>
17. He, Y., Ling, S., Tang, K.H., Wang, H.: Everlasting fully dynamic group signatures. In: ACNS (3). Lecture Notes in Computer Science, vol. 15827, pp. 3–28. Springer (2025)
18. Hesse, J., Singh, N., Sorniotti, A.: How to bind anonymous credentials to humans. In: USENIX Security Symposium. pp. 3047–3064. USENIX Association (2023)
19. Hülsing, A., Kudinov, M.A., Ronen, E., Yorgev, E.: SPHINCS+C: compressing SPHINCS+ with (almost) no cost. In: SP. pp. 1435–1453. IEEE (2023)
20. Kogan, D., Manohar, N., Boneh, D.: T/key: Second-factor authentication from secure hash chains. In: CCS. pp. 983–999. ACM (2017)
21. Kumar, V., Li, H., Park, J.J., Bian, K., Yang, Y.: Group signatures with probabilistic revocation: A computationally-scalable approach for providing privacy-preserving authentication. In: CCS. pp. 1334–1345. ACM (2015)
22. Lamport, L.: Password authentication with insecure communication. Commun. ACM **24**(11), 770–772 (1981)
23. Liu, J.K., Chu, C., Chow, S.S.M., Huang, X., Au, M.H., Zhou, J.: Time-bound anonymous authentication for roaming networks. IEEE Trans. Inf. Forensics Secur. **10**(1), 178–189 (2015)
24. M'Raihi, D., Machani, S., Pei, M., Rydell, J.: Totp: Time-based one-time password algorithm (2011), <https://tools.ietf.org/html/rfc6238>
25. M'Raihi, D., Bellare, M., Hoornaert, F., Naccache, D., Ranen, O.: HOTP: an hmac-based one-time password algorithm. RFC **4226**, 1–37 (2005)
26. Murmann, J.P., Korn, J., Worch, H.: How fast can firms grow? Jahrbücher für Nationalökonomie und Statistik **234**(2-3), 210–233 (2014)
27. Nisbet, R., Miner, G., Yale, K.: Chapter 10 - numerical prediction. In: Nisbet, R., Miner, G., Yale, K. (eds.) Handbook of Statistical Analysis and Data Mining Applications (Second Edition), pp. 187–213. Academic Press, Boston, second edition edn. (2018)
28. Odelu, V., Das, A.K., Goswami, A.: A secure biometrics-based multi-server authentication protocol using smart cards. IEEE Trans. Inf. Forensics Secur. **10**(9), 1953–1966 (2015)
29. Rodríguez, J.G., Krenn, S., Slamanig, D.: To pass or not to pass: Privacy-preserving physical access control. Comput. Secur. **136**, 103566 (2024)
30. Sun, H., Sun, K., Wang, Y., Jing, J.: Trustotp: Transforming smartphones into secure one-time password tokens. In: CCS. pp. 976–988. ACM (2015)
31. Tolosana, R., Vera-Rodríguez, R., Fierrez, J.: Biotouchpass: Handwritten passwords for touchscreen biometrics. IEEE Trans. Mob. Comput. **19**(7), 1532–1543 (2020)
32. Wheat, C., Farrell, D.: The ups and downs of small business employment: Big data on payroll growth and volatility. JPMorgan Chase & Co. Institute (2017)
33. Yang, Q., Xue, K., Xu, J., Wang, J., Li, F., Yu, N.: Anfra: Anonymous and fast roaming authentication for space information network. IEEE Trans. Inf. Forensics Secur. **14**(2), 486–497 (2019)
34. Yang, Z., Jin, C., Ning, J., Li, Z., Dinh, A., Zhou, J.: Group time-based one-time passwords and its application to efficient privacy-preserving proof of location. In: ACSAC. pp. 497–512. ACM (2021)

35. Zeng, M., Cui, J., Zhang, Q., Zhong, H., He, D.: Efficient revocable cross-domain anonymous authentication scheme for iiot. *IEEE Trans. Inf. Forensics Secur.* **20**, 996–1010 (2025)
36. Zhang, Q., Wu, J., Zhong, H., He, D., Cui, J.: Efficient anonymous authentication based on physically unclonable function in industrial internet of things. *IEEE Trans. Inf. Forensics Secur.* **18**, 233–247 (2023)

A Cryptographic Building Blocks

Time-based One-time Passwords. We adopt an asymmetric TOTP scheme similar to that in GTOTP, defined by four algorithms (**Setup**, **Plnit**, **PGen**, **Verify**). **Setup**($1^\kappa, T_s, T_e, \Delta_{pw}$) outputs the password number $N = (T_e - T_s) / \Delta_{pw}$ given the security parameter, start and end times T_s and T_e , and the password generation interval Δ_{pw} . **Plnit**(sd) takes a secret seed $sd \in \mathcal{K}_{\text{TOTP}}$ and outputs a verification point $vp \in \mathcal{VP}_{\text{TOTP}}$. **PGen**(sd, T) takes a seed sd and time T , and outputs a password $pw \in \mathcal{PW}_{\text{TOTP}}$ associated with the time T . **Verify**(vp, pw, T) checks the validity of password pw at time T against vp , outputting 1 if valid and 0 otherwise. We require that the verification point in TOTP can be computed based on the secret seed or password. We define a security game $G_{\mathcal{A}, \text{TOTP}}^{\text{TOTP Forge}}(\kappa, T_s, T_e, \Delta_{pw})$ for a time-based one-time passwords scheme TOTP in Figure 6, with a key space $\mathcal{K}_{\text{TOTP}}$ for the input secret seed, a verification point space $\mathcal{VP}_{\text{TOTP}}$, and a password space $\mathcal{PW}_{\text{TOTP}}$. We assume that the challenger keeps a system clock T_{ct} which is updated every Δ_{pw} . The goal of adversary in the game is to forge a valid password of TOTP for a future time.

$G_{\mathcal{A}, \text{TOTP}}^{\text{TOTP Forge}}(\kappa, T_s, T_e, \Delta_{pw}) :$	
Initialize (T_s, T_e, Δ_{pw}) : $\text{pms}_{\text{TOTP}} \leftarrow \text{TOTP.Setup}(1^\kappa, T_s, T_e, \Delta_{pw})$ $sd \xleftarrow{\$} \mathcal{K}_{\text{TOTP}}$ $vp \leftarrow \text{TOTP.Plnit}(sd)$ OUTPUT $\text{pms}_{\text{TOTP}}, vp$	Finalize () : IF $\exists (pw^*, T^*) \in \text{HD}$ s.t. $(\text{Verify}(vp, pw^*, T^*) = 1$ and no GetNextPw () at $\tilde{T}$ s.t. $\tilde{T} \geq T^*$ OUTPUT 1 OUTPUT 0
GetNextPw () : OUTPUT $\text{TOTP.PGen}(sd, T_c)$	ReceivePw (pw) : APPEND $(pw, T_c) \rightarrow \text{HD}$ OUTPUT $\text{TOTP.Verify}(vp, pw, T_c)$

Fig. 6: Procedures Used to Define Security for TOTP.

Definition 2. We say a TOTP protocol is secure if no PPT adversary has a non-negligible advantage $\text{Adv}_{\mathcal{A}, \text{TOTP}}^{\text{TOTP Forge}}(\kappa, T_s, T_e, \Delta_{pw}) := \Pr \left[G_{\mathcal{A}, \text{TOTP}}^{\text{TOTP Forge}}(\kappa, T_s, T_e, \Delta_{pw}) = 1 \right]$ with given parameters.

Pseudo-random Function Family. A pseudo-random function (PRF) family consists of (**Setup**, **Eval**). **Setup**(1^κ) takes as input the security parameter 1^κ ,

and outputs a secret key $k \xleftarrow{\$} \mathcal{K}_{\text{PRF}}$ and system parameter pms_{PRF} . $\text{Eval}(k, x)$ is a deterministic algorithm that takes as input the secret key k and a message $x \in \mathcal{M}_{\text{PRF}}$, and outputs the evaluation result $r \in \mathcal{R}_{\text{PRF}}$. We define a security game $G_{\mathcal{A}, \text{F}}^{\text{PRF}}(\kappa, q_f)$ (see also in Figure 7) that is played between a probabilistic polynomial time (PPT) adversary $\mathcal{A}$ and a challenger based on F and the security parameter κ .

$G_{\mathcal{A}, \text{F}}^{\text{PRF}}(\kappa, q_f) :$	
<u>Initialize()</u> :	<u>Finalize(b^*) :</u>
$(\text{pms}_{\text{PRF}}, k) \leftarrow \text{F.Setup}(1^\kappa)$	IF $b^* = b$ and $x^* \notin \text{FL}$, OUTPUT 1
OUTPUT pms_{PRF}	ELSE OUTPUT 0
<u>Challenge(x^*) :</u>	<u>FuncQ(x) :</u>
$b \xleftarrow{\$} \{0, 1\}; r_0 \xleftarrow{\$} \mathcal{R}_{\text{PRF}}; r_1 \leftarrow \text{F.Eval}(k, x^*)$	APPEND $x \rightarrow \text{FL}$
OUTPUT r_b	OUTPUT $\text{F.Eval}(k, x)$

Fig. 7: Procedures Used to Define Security for PRF.

Definition 3. We say F is secure if the advantage $\text{Adv}_{\mathcal{A}, \text{F}}^{\text{PRF}}(\kappa, q_f) := \left| \Pr[G_{\mathcal{A}, \text{F}}^{\text{PRF}}(\kappa, q_f) = 1] - \frac{1}{2} \right|$ of any PPT adversary $\mathcal{A}$ is negligible under κ .

Authenticated Symmetric Encryption. An authenticated symmetric encryption scheme ASE consists of three algorithms (Setup , Enc , Dec). $\text{Setup}(1^\kappa)$ takes the security parameter and outputs a secret key $k \in \mathcal{K}_{\text{ASE}}$ and system parameter pms_{ASE} . $\text{Enc}(k, m)$ takes a secret key k and message $m \in \mathcal{M}_{\text{ASE}}$ and outputs ciphertext $C \in \mathcal{C}_{\text{ASE}}$. $\text{Dec}(k, C)$ takes a key k and ciphertext C , and outputs the corresponding message m . We say an ASE scheme is randomized if Enc is a probabilistic (randomized) algorithm. We define a security game $G_{\mathcal{A}, \text{ASE}}^{\text{IND-CCA}}(\kappa, q_e)$ in Figure 8 to formulate the standard notion of indistinguishability under chosen-ciphertext attacks (IND-CCA), that is played between a PPT adversary $\mathcal{A}$ and a challenger based on ASE and κ .

$G_{\mathcal{A}, \text{ASE}}^{\text{IND-CCA}}(\kappa, q_e) :$		
<u>Initialize()</u> :	<u>Finalize(b^*) :</u>	<u>Challenge(m_0, m_1) :</u>
$(\text{pms}, k) \leftarrow \text{ASE.Setup}(1^\kappa)$	IF $b^* = b$ and $C^* \notin \text{CL}$	$b \xleftarrow{\$} \{0, 1\}$
OUTPUT pms	OUTPUT 1	$C^* \leftarrow \text{ASE.Enc}(k, m_b)$
	OUTPUT 0	OUTPUT C^*
<u>DecP(C) :</u>	<u>EncP(m) :</u>	
APPEND $C \rightarrow \text{CL}$	OUTPUT $\text{ASE.Enc}(k, m)$	
OUTPUT $\text{ASE.Dec}(k, C)$		

Fig. 8: Procedures Used to Define Security for ASE.

Definition 4. We say ASE is secure if for any PPT adversaries, the advantage $\text{Adv}_{\mathcal{A}, \text{ASE}}^{\text{IND-CCA}}(\kappa, q_e) := \left| \Pr[G_{\mathcal{A}, \text{ASE}}^{\text{IND-CCA}}(\kappa, q_e) = 1] - \frac{1}{2} \right|$ is negligible under κ .

Collision-resistant Hash Functions. A collision-resistant hash function CRH consists of two algorithms (Setup, Eval). $\text{Setup}(1^\kappa)$ takes the security parameter and outputs a key $hk \in \mathcal{K}_{\text{CRH}}$ and system parameter pms_{CRH} . $\text{Eval}(hk, m)$ takes a key hk and a message $m \in \mathcal{M}_{\text{CRH}}$ and outputs a hash value $y \in \mathcal{R}_{\text{CRH}}$. The CRH security game $G_{\mathcal{A}, \text{H}}^{\text{CRH}}(\kappa)$ based on an adversary $\mathcal{A}$ and a CRH family H is defined in Figure 9, where H has key space $\mathcal{K}_{\text{CRH}}$, message space $\mathcal{M}_{\text{CRH}}$ and hash value space $\mathcal{Y}_{\text{CRH}}$.

$G_{\mathcal{A}, \text{H}}^{\text{CRH}}(\kappa) :$	
Initialize() :	Finalize(m, m') :
$(\text{pms}_{\text{CRH}}, hk) \leftarrow \text{H.Setup}(1^\kappa)$	IF $m \neq m'$ and $\text{H.Eval}(hk, m) = \text{H.Eval}(hk, m')$
OUTPUT hk	OUTPUT 1
	OUTPUT 0

Fig. 9: Procedures Used to Define Security for CRH.

Definition 5. We say H is secure if no PPT adversary has non-negligible advantage $\text{Adv}_{\mathcal{A}, \text{H}}^{\text{CRH}}(\kappa) := \Pr[G_{\mathcal{A}, \text{H}}^{\text{CRH}}(\kappa) = 1]$ under κ .

Merkle Tree. A Merkle tree is constructed from a series of data items, which form the leaves of a Merkle tree. It can act as an authenticated data structure for efficient verification of a huge data set. We define a Merkle tree scheme with three algorithms (Build, GetPf, Verify). $\text{Build}(\{\text{Lf}_i\}_{i \in [\ell]})$ takes a list of data items as leaves and builds a Merkle tree MTI, whose internal nodes are computed as $\text{H}(\text{node.LeftChild} \parallel \text{node.RightChild})$. $\text{GetPf}(\text{MTI}, \text{Lf}_i)$ takes the tree MTI and a leaf Lf_i , and outputs an inclusion proof PfLf_i consisting of the sibling hashes along the path to the root. $\text{Verify}(\text{MTI}, \text{Lf}_i, \text{PfLf}_i)$ takes the root, a leaf, and its proof, and outputs 1 if the recomputed root matches MTI and 0 otherwise. We define a security game $G_{\mathcal{A}, \text{MT}}^{\text{MT-Forge}}(\kappa)$ in Figure 10 for a Merkle tree scheme MT, which encompasses the following procedures. For a secure Merkle tree scheme, the adversary must not forge the Merkle proof for a leaf node which does not belong to the Merkle tree.

$G_{\mathcal{A}, \text{MT}}^{\text{MT-Forge}}(\kappa) :$	
Initialize($\{\text{Lf}_i\}_{i \in [\ell]}$) :	Finalize(Lf^*, Pf^*) :
$\text{MTI} \leftarrow \text{MT.Build}(\{\text{Lf}_i\}_{i \in [\ell]})$	IF $1 \leftarrow \text{MT.Verify}(\text{Rt}, \text{Lf}^*, \text{Pf}^*)$
OUTPUT MTI	and $\text{Lf}^* \notin \{\text{Lf}_i\}_{i \in [\ell]}$, OUTPUT 1
	OUTPUT 0

Fig. 10: Procedures Used to Define Security for MT.

Definition 6. We say MT is secure if no PPT adversary has non-negligible advantage $\text{Adv}_{\mathcal{A}, \text{MT}}^{\text{MT-Forge}}(\kappa) := \Pr[G_{\mathcal{A}, \text{MT}}^{\text{MT-Forge}}(\kappa) = 1]$ under κ .

Unpredictable Permutation. We define a keyed permutation as two algorithms (Setup, Shuffle). Let $\mathcal{K}_{\text{PM}}$ be the key space, $\mathcal{M}_{\text{PM}}$ be the element space. Setup(1^κ) takes the security parameter and outputs a key $k \in \mathcal{K}_{\text{PM}}$ and system parameter pms_{PM} . Shuffle(k, X) takes as input the permutation key k and a set of elements $X = \{x_i\}_{i \in [n]}$, where $X \in \mathcal{M}_{\text{PM}}$ is of size $n \in \mathbb{N}$, and outputs a permuted set $Y \in \mathcal{M}_{\text{PM}}$. In Figure 11, we define a security game $G_{\mathcal{A}, \text{PM}}^{\text{UP}}(\kappa)$ that is played between a PPT adversary $\mathcal{A}$ and a challenger based on a permutation scheme PM and the security parameter κ , where PM has the key space $\mathcal{K}_{\text{PM}}$ and the element space $\mathcal{M}_{\text{PM}}$.

$G_{\mathcal{A}, \text{PM}}^{\text{UP}}(\kappa) :$		
Initialize(n) :		Finalize(β^*) :
$(\text{pms}_{\text{PM}}, k) \leftarrow \text{PM.Setup}(1^\kappa, n)$		IF $\beta^* = \beta_0$, OUTPUT 1
OUTPUT pms_{MT}		OUTPUT 0
Challenge($\{x_i\}_{i \in [n]}, w_0, w_1$) :		
IF $w_0, w_1 \notin [n]$ or $w_0 = w_1$ OUTPUT $\perp$		
$\{y_i\}_{i \in [n]} := \text{PM.Shuffle}(k, \{x_i\}_{i \in [n]})$		
Let β_0 and β_1 be indices s.t. $y_{\beta_0} = x_{w_0}$ and $y_{\beta_1} = x_{w_1}$		
OUTPUT $\{y_i\}_{i \in [n] \setminus \{\beta_0, \beta_1\}}$		

Fig. 11: Procedures Used to Define Security for PM.

Definition 7. We say PM is secure if for any PPT adversaries, the advantage $\text{Adv}_{\mathcal{A}, \text{PM}}^{\text{UP}}(\kappa) := \left| \Pr[G_{\mathcal{A}, \text{PM}}^{\text{UP}}(\kappa) = 1] - \frac{1}{2} \right|$ is negligible under κ .

Digital Signature. A signature scheme Sig consists of three algorithms (Setup, Sign, Verify). Setup(1^κ) takes the security parameter and outputs a key pair $(sk, pk) \in \mathcal{K}_{\text{Sig}}$ and system parameter pms_{Sig} . Sign(sk, m) takes a secret key and message $m \in \{0, 1\}^*$ and outputs a signature σ . Verify(pk, m, σ) takes the public key, message, and signature, and outputs 1 if the signature is valid and 0 otherwise. We define a security game $G_{\mathcal{A}, \text{Sig}}^{\text{EU-CMA}}(\kappa, q_s)$ (see also in Figure 12) that is played between a PPT adversary $\mathcal{A}$ and a challenger based on the signature scheme Sig and the security parameter κ .

$G_{\mathcal{A}, \text{Sig}}^{\text{EU-CMA}}(\kappa, q_s) :$		
Initialize() :		SignOracle(m) :
$(pk, sk) \leftarrow \text{Sig.Setup}(1^\kappa)$		$\sigma \leftarrow \text{Sig.Sign}(sk, m)$
OUTPUT pk		APPEND $m \rightarrow \text{ML}$
		OUTPUT σ
		Finalize(m^*, σ^*) :
		IF $\text{Sig.Verify}(pk, m^*, \sigma^*) = 1$
		and $m^* \notin \text{ML}$
		OUTPUT 1
		ELSE OUTPUT 0

Fig. 12: Procedures Used to Define Security for Sig.

Definition 8. We say a digital signature scheme Sig is secure if for every PPT adversary $\mathcal{A}$ making at most q_s signing queries, the advantage $\text{Adv}_{\mathcal{A}, \text{Sig}}^{\text{EU-CMA}}(\kappa, q_s) := \Pr[G_{\mathcal{A}, \text{Sig}}^{\text{EU-CMA}}(\kappa, q_s) = 1]$ is negligible under κ .

B NWDGT

We now present the details of NWDGT and its security analysis.

B.1 Detailed Algorithms of NWDGT

We use a SPHINCS+C construction without the FORS+C layer, i.e., only the hyper-tree structure, as the digital signature HT below. The proposed NWDGT is described as follows.

- **Setup**($1^\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e$): RA samples a secret key k_{RA} , sets T_s to the current time T_c , and computes the number of TOTP instances $E := (T_e - T_s)/\Delta_{vp}$ and the number of passwords per instance $N := (T_e - T_s)/\Delta_{pw}$. Next, RA initializes the building blocks including a pseudo-random function F as $\text{pms}_{\text{PRF}} := F.\text{Setup}(1^\kappa)$, an authenticated symmetric encryption scheme ASE as $\text{pms}_{\text{ASE}} := \text{ASE}.\text{Setup}(1^\kappa)$, a collision-resistant hash function H as $\text{pms}_{\text{CRH}} := H.\text{Setup}(1^\kappa)$, and a permutation PM as $\text{pms}_{\text{PM}} := \text{PM}.\text{Setup}(1^\kappa)$. RA generates HT public key pk_{HT} by computing $(\text{pms}_{\text{HT}}, pk_{\text{HT}}, sk_{\text{HT}}) \leftarrow \text{HT}.\text{Setup}(1^\kappa)$, where the secret key sk_{HT} of HT scheme is also generated, and sets the group public key $\text{gpk}_G := pk_{\text{HT}}$.

Meanwhile, RA maintains the group management information GI_G , which includes a registration list IDL_G for storing member identities, along with a set of variables $\{\text{RS}_{s_j^i} := C_{s_j^i}^i\}_{j \in [U]}$ corresponding to currently active verify epochs. These variables are periodically updated by RA to remove expired entries and prepare for the upcoming epoch. RA also manages a revocation list RL_G , a bit array of length U , where each bit, initialized to 0, indicates whether the corresponding member has been revoked.

- **MInit**(ID_j): This algorithm is identical to the one in LWDGT, as described in Section 5.2.
- **Join**($k_{\text{RA}}, \text{gpk}_G, \text{ID}_j, \text{vps}_{\text{ID}_j}$): During the i -th epoch, RA continuously receives verification points from different members and maintains a set of size ids_{i+1} , representing the identities of members registered for the next epoch. Before the end of the $(i - 1)$ -th epoch, RA should generate auxiliary information $\text{Ax}_{\text{ID}_j} = (\sigma_{\text{ID}_j}^i, C_{\text{ID}_j}^i)$ for the i -th epoch and return it to the group member through the following steps: (i) compute the i -th ASE key $k_{\text{ASE}}^i := F(k_{\text{RA}}, \text{"ASE"} \| i)$, the identity ciphertext $C_{\text{ID}_j}^i := \text{ASE}.\text{Enc}(k_{\text{ASE}}^i, \text{ID}_j)$ and the updated verification point $\hat{vp}_{\text{ID}_j}^i := H(vp_{\text{ID}_j}^i \| C_{\text{ID}_j}^i \| i)$ for the member's requested registration epochs; (ii) compute the i -th permutation key k_{PM}^i and shuffle S_i to generate a permuted set $S'_i := \text{PM}.\text{Shuffle}(k_{\text{PM}}^i, S_i)$, where S_i denotes the updated verification points of all registered members in the i -th epoch; (iii) build a Merkle tree $\text{MT}_i := \text{MT}.\text{Build}(S'_i)$ and compute the Merkle proof $\text{Pf}_{\hat{vp}_{\text{ID}_j}^i} := \text{MT}.\text{GetPf}(\text{MT}_i, \hat{vp}_{\text{ID}_j}^i)$;

(iv) compute the HT signature $\sigma_{\text{HT}}^i := \text{HT.Sign}(sk_{\text{HT}}, \text{MT}_i)$ and return $\sigma_{\text{ID}_j}^i := (\text{Pf}_{\hat{v}p_{\text{ID}_j}^i}, \sigma_{\text{HT}}^i)$.

- $\text{GetSD}(k_{\text{ID}_j}, T)$: This algorithm is also identical to that of LWDGT (given in Section 5.2).

- $\text{PwdGen}(k_{\text{ID}_j}, T)$: The party ID_j first computes the index i as $i := \frac{T-T_s}{\Delta_{vp}}$. Next, it computes the secret seed $sd_{\text{ID}_j}^i := F(k_{\text{ID}_j}, \text{ID}_j \| i)$ and the password $\bar{p}w_{\text{ID}_j}^{i,z} := \text{TOTP.PGen}(sd_{\text{ID}_j}^i, T_c)$, where $z := \frac{T_c - T_s - i \cdot \Delta_{vp}}{\Delta_{pw}}$ is the index of password in the i -th epoch. Then, the password $pw_{\text{ID}_j}^{i,z} := (\bar{p}w_{\text{ID}_j}^{i,z}, \text{Pf}_{\hat{v}p_{\text{ID}_j}^i}, \sigma_{\text{HT}}^i, C_{\text{ID}_j}^i)$ is returned.

- $\text{Verify}(\text{gpk}_G, pw_{\text{ID}_j}^{i,z}, T, \text{RL}_G)$: The verifier first gets the value of idx such that RS_{idx} contains the identity ciphertext $C_{\text{ID}_j}^i$. If $\text{RL}_G[idx] = 1$, it outputs 0. Otherwise, it computes the verification point $vp_{\text{ID}_j}^i$ from $\bar{p}w_{\text{ID}_j}^{i,z}$ and T , and $\hat{v}p_{\text{ID}_j}^i := H(vp_{\text{ID}_j}^i \| C_{\text{ID}_j}^i \| i)$. Then, it outputs 1, if and only if $\text{TOTP.Verify}(vp_{\text{ID}_j}^i, \bar{p}w_{\text{ID}_j}^{i,z}, T) = 1$ and $\text{HT.Verify}(\text{gpk}_G, Rt, \sigma_{\text{HT}}^i) = 1$, where the tree root Rt is computed based on $\hat{v}p_{\text{ID}_j}^i$ and $\text{Pf}_{\hat{v}p_{\text{ID}_j}^i}$.

- $\text{Revoke}(k_{\text{RA}}, \text{ID}_j, \text{gpk}_G, \text{RL}_G, \text{Gl}_G)$: The algorithm is the same as that of LWDGT (see Section 5.2).

- $\text{IDOpen}(k_{\text{RA}}, \text{gpk}_G, pw_{\text{ID}_j}^{i,z}, T)$: This algorithm is identical to the one in LWDGT, as given in Section 5.2.

B.2 Security Analysis of NWDGT

Theorem 3. *Suppose that the building blocks TOTP, H, PRF, HT, ASE, MT and PM are secure, then NWDGT provides anonymity.*

Proof. We prove the claim using a sequence-of-games approach, starting from the original Anony experiment and gradually modifying it until the adversary's advantage becomes negligible. For each game i , let $\text{BK}_i^{\text{Anony}}$ denote the event that the adversary wins (i.e., the experiment outputs 1).

Game 0. This game is the real Anony game. By definition, we have

$$\Pr[\text{BK}_0^{\text{Anony}}] = 1/2 + \text{Adv}_{\mathcal{A}, \text{NWDGT}}^{\text{Anony}}(\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e).$$

Game 1. The challenger aborts if two independent executions of PRF.Setup yield the same key. Such collisions occur only with negligible probability; otherwise, an adversary could break the PRF security directly. More concretely, suppose the PRF challenger provides a challenge key k^* while the adversary samples $U - 1$ additional keys. If any two keys collide, then with probability at least $2/U$, k^* is among them, exposing it and violating the PRF assumption. Therefore,

$$\Pr[\text{BK}_0^{\text{Anony}}] \leq \Pr[\text{BK}_1^{\text{Anony}}] + \frac{U}{2} \cdot \text{Adv}_{\mathcal{A}, \text{PRF}}^{\text{PRF}}(\kappa, 1).$$

Game 2. We introduce an abort condition: the challenger aborts if it fails to correctly guess the two honest members (among U) and the challenge verification epoch (among E) used in the **Challenge** query. Since the guess is uniform, the success probability is bounded by $1/(2U \cdot E)$. Thus,

$$\Pr[\text{BK}_1^{\text{Anony}}] = 2U \cdot E \cdot \Pr[\text{BK}_2^{\text{Anony}}].$$

Game 3. In this game, we modify the values derived from the PRF at the guessed challenge epoch i^* . Specifically, the set $\{k_{\text{PM}}^{i^*}, k_{\text{ASE}}^{i^*}, sd_{\alpha_{\text{ID}_0}}^{i^*}, sd_{\alpha_{\text{ID}_1}}^{i^*}\}$ is replaced by four independently and uniformly chosen random strings of the same length. The rest of the experiment remains identical. To argue that this transition is indistinguishable, we rely on a hybrid argument. We define a sequence of hybrid experiments H_0, H_1, H_2, H_3, H_4 , where H_0 corresponds to the previous game in which all four values are PRF outputs, and in each subsequent hybrid one additional element from the set is replaced by a random string. Thus, H_4 coincides with the current game. Consider any two consecutive hybrids H_{j-1} and H_j . If an adversary $\mathcal{D}$ can distinguish between them, we construct a reduction $\mathcal{B}_j$ to break the security of the underlying PRF. The reduction $\mathcal{B}_j$ is given oracle access to a challenger that on input a label either returns a true PRF evaluation under a hidden key or an independent uniform string. In simulating the scheme for $\mathcal{D}$, $\mathcal{B}_j$ embeds the challenger's response exactly at the position of the j -th element in the set for epoch i^* . All other PRF outputs required in the simulation are computed locally by $\mathcal{B}_j$ using independently sampled keys, ensuring that the entire execution is consistent with the scheme specification. If the challenger returns a real PRF evaluation, the view of $\mathcal{D}$ is distributed exactly as in H_{j-1} ; if the challenger instead provides a random string, the resulting view is that of H_j . Therefore, any distinguishing advantage of $\mathcal{D}$ between the two hybrids transfers directly to $\mathcal{B}_j$ as an advantage against the PRF. Summing over all four replacements, the cumulative change in the adversary's success probability is at most four times the PRF advantage. Formally, we obtain

$$\Pr[\text{BK}_2^{\text{Anony}}] \leq \Pr[\text{BK}_3^{\text{Anony}}] + 4 \cdot \text{Adv}_{\mathcal{A}, \text{PRF}}^{\text{PRF}}(\kappa, E).$$

Game 4. Here, the challenge ciphertext in the **Challenge** query is always generated using ID_0 , regardless of the challenge bit b . If an adversary can distinguish this game from the previous one, we construct an adversary $\mathcal{C}$ that breaks the IND-CCA security of ASE. Specifically, $\mathcal{C}$ forwards the pair $(\alpha_{\text{ID}_0}, \alpha_{\text{ID}_1})$ to the ASE challenger and receives a challenge ciphertext. If the ciphertext encrypts α_{ID_0} , it matches the previous game; if α_{ID_1} , it matches this game. Thus,

$$\Pr[\text{BK}_3^{\text{Anony}}] \leq \Pr[\text{BK}_4^{\text{Anony}}] + \text{Adv}_{\mathcal{A}, \text{ASE}}^{\text{IND-CCA}}(\kappa, E).$$

The changed game implies that the adversary cannot infer any information regarding the identity from the ASE ciphertext.

Game 5. Finally, instead of using the permutation mechanism, the challenger chooses uniformly at random between the two leaves intended for ID_0 and ID_1 . An adversary that distinguishes this modification can break the unpredictability

of PM. The reduction queries the PM challenger on the set $\{1, \dots, U\}$ with two positions corresponding to the challenge members' join order. If the output aligns with the chosen leaf, the game is simulated as before; otherwise, it matches the modified game. The adversary's decision predicts PM's output, contradicting its security assumption. Hence,

$$\Pr[\text{BK}_4^{\text{Anony}}] \leq \Pr[\text{BK}_5^{\text{Anony}}] + \text{Adv}_{\mathcal{A}, \text{PM}}^{\text{UP}}(\kappa).$$

In this final game, the adversary cannot distinguish which identity corresponds to which verification point. Even if all group members except ID_0 and ID_1 are corrupted, these two honest members occupy indistinguishable positions selected uniformly at random. Therefore, $\Pr[\text{BK}_5^{\text{Anony}}] = \frac{1}{2}$.

Combining all the bounds across the game transitions, the adversary's overall advantage is negligible. This completes the proof of Theorem 3.

Theorem 4. *Under the same assumptions as in Theorem 3, NWDGT provides traceability.*

Proof. Let $\text{BK}_i^{\text{Trace}}$ denote the event that the i -th instance of the (modified) Trace game outputs 1.

Game 0. This is the original Trace game. By definition,

$$\Pr[\text{BK}_0^{\text{Trace}}] = \text{Adv}_{\mathcal{A}, \text{NWDGT}}^{\text{Trace}}(\kappa, \mathbf{G}, \Delta_{pw}, \Delta_{vp}, T_s, T_e).$$

Game 1. As before, the challenger aborts if any two honest group members share the same PRF key. By the same argument as in Game 1 of Theorem 3, we obtain

$$\Pr[\text{BK}_0^{\text{Trace}}] \leq \Pr[\text{BK}_1^{\text{Trace}}] + \frac{U}{2} \cdot \text{Adv}_{\mathcal{A}, \text{F}}^{\text{PRF}}(\kappa, 1).$$

Game 2. Let vp^* be the verification point corresponding to the password pw^* submitted in the Finalize query. The challenger aborts if there exist indices i^* , u such that $\text{H}(vp^* || C_{\text{ID}_j^*} || i^*) = \text{H}(vp_{\text{ID}_j^*}^u || C_{\text{ID}_j^*}^u || u)$ and ID^* is registered by AddHM. If the adversary causes this abort with non-negligible probability, we can construct an algorithm that breaks the collision resistance of H . Therefore,

$$\Pr[\text{BK}_1^{\text{Trace}}] \leq \Pr[\text{BK}_2^{\text{Trace}}] + \text{Adv}_{\mathcal{A}, \text{H}}^{\text{CRH}}(\kappa).$$

Game 3. This game is identical to the previous one, except that the challenger aborts if the adversary outputs a password $\bar{pw}_{\text{ID}_j^*}^{*, i, z} := (\bar{pw}_{\text{ID}_j^*}^{*, i, z}, \text{Pf}_{vp_{\text{ID}_j^*}^*}^{i, z}, \sigma_{\text{HT}}^i, C_{\text{ID}^*}^i)$ for some T^* such that: i) $\text{Verify}(\text{gpk}_G, \bar{pw}_{\text{ID}_j^*}^{*, i, z}, T^*, \text{RL}_G) = 1$; and ii) $\sigma_{\text{ID}^*}^i$ was not produced by the challenger during AddHM or AddMM. If this happens with non-negligible probability, one can break the existential unforgeability under chosen message attack (EU-CMA) of HT and the unforgeability of MT. Hence,

$$\Pr[\text{BK}_2^{\text{Trace}}] \leq \Pr[\text{BK}_3^{\text{Trace}}] + \text{Adv}_{\mathcal{A}, \text{HT}}^{\text{EU-CMA}}(\kappa, E) + \text{Adv}_{\mathcal{A}, \text{MT}}^{\text{MT_Forge}}(\kappa).$$

Game 4. The challenger aborts if it cannot correctly guess the owner ID^* and the verification epoch i^* of the forged password. The guessing probability is at

most $1/(U \cdot E)$, so $\Pr[\text{BK}_3^{\text{Trace}}] = U \cdot E \cdot \Pr[\text{BK}_4^{\text{Trace}}]$. We denote i^* as the guessed verification epoch and ID^* as the target uncorrupted member.

Game 5. Similar to the anonymity proof, the outputs of F at epoch i^* , namely $\{k_{\text{PM}}^{i^*}, k_{\text{ASE}}^{i^*}, sd_{\alpha_{\text{ID}_0}}^{i^*}, sd_{\alpha_{\text{ID}_1}}^{i^*}\}$, are replaced with truly random values. Applying the same hybrid argument yields

$$\Pr[\text{BK}_4^{\text{Trace}}] \leq \Pr[\text{BK}_5^{\text{Trace}}] + 4 \cdot \text{Adv}_{\mathcal{A}, F_1}^{\text{PRF}}(\kappa, E).$$

Game 6. Finally, the challenger aborts if the adversary outputs a valid password $\bar{p}w^*$ for ID^* at some time T^* beyond all previously generated password values. Such a forgery would break the unforgeability of TOTP. Constructing an adversary $\mathcal{B}$ that simulates this modified game and uses $\text{GetNextPw}()$ appropriately yields

$$\Pr[\text{BK}_5^{\text{Trace}}] \leq \Pr[\text{BK}_6^{\text{Trace}}] + \text{Adv}_{\mathcal{A}, \text{TOTP}}^{\text{TOTP_Forge}}(\kappa, T_s, T_e, \Delta_{pw}).$$

At this point, the adversary cannot forge any password of an uncorrupted member with non-negligible probability, so the advantage is zero. This completes the proof.

C Pseudocodes of LWDGT

Here, we provide the pseudocodes for the member join procedure Join and verification procedure Verify of LWDGT.

Algorithm 1: Join: Member Join Procedure

Input: $k_{\text{RA}}, \text{gpk}_G, \text{ID}_j, \text{vps}_{\text{ID}_j}$
Output: $\text{Ax}_{\text{ID}_j} = (\sigma_{\text{ID}_j}^i, C_{\text{ID}_j}^i)$ where $\sigma_{\text{ID}_j}^i := (\sigma_{\text{OTS}}^i, \text{Pf}_{\text{ID}_j}^i, \sigma_{\text{SPHINCS+C}}^i)$

/ Step 1: Identity ciphertext and updated verification point generation */*

- 1 $k_{\text{ASE}}^i \leftarrow F(k_{\text{RA}}, \text{"ASE"} \parallel i);$
- 2 $C_{\text{ID}_j}^i \leftarrow \text{ASE.Enc}(k_{\text{ASE}}^i, \text{ID}_j);$ // Randomized encryption, $C_{\text{ID}_j}^i \neq C_{\text{ID}_j}^{i+1}$
- 3 $\hat{v}p_{\text{ID}_j}^i \leftarrow H(vp_{\text{ID}_j}^i \parallel C_{\text{ID}_j}^i \parallel i);$

/ Step 2: OTS key generation */*

- 4 Find idx such that $s_{\text{idx}}^i = \alpha_{\text{ID}_j}$;
- 5 Generate l OTS key pairs $\{(pk_j^i, sk_j^i)\}$ for $j \in [l(\lceil \text{idx}/l \rceil - 1) + 1, \dots, l\lceil \text{idx}/l \rceil]$ using $\text{OTS.Setup}(1^\kappa)$;

/ Step 3: Build OTS-tree */*

- 6 $P_{\lceil \text{idx}/l \rceil} \leftarrow \{pk_j^i\}_{j \in [l(\lceil \text{idx}/l \rceil - 1) + 1, \dots, l\lceil \text{idx}/l \rceil]}$;
- 7 $\text{MT}_{\lceil \text{idx}/l \rceil}^i \leftarrow \text{MT.Build}(P_{\lceil \text{idx}/l \rceil}^i);$

/ Step 4: Signature and proof generation */*

- 8 $\sigma_{\text{OTS}}^i \leftarrow \text{OTS.Sign}(sk_{\text{idx}}^i, \hat{v}p_{\text{ID}_j}^i);$
- 9 $\text{Pf}_{\text{ID}_j}^i \leftarrow \text{MT.GetPf}(\text{MT}_{\lceil \text{idx}/l \rceil}^i, pk_{\text{idx}}^i);$
- 10 $\sigma_{\text{SPHINCS+C}}^i \leftarrow \text{SPHINCS+C.Sign}(sk_{\text{SPHINCS+C}}, \text{MT}_{\lceil \text{idx}/l \rceil}^i);$
- 11 **return** $(\sigma_{\text{ID}_j}^i, C_{\text{ID}_j}^i)$ with $\sigma_{\text{ID}_j}^i := (\sigma_{\text{OTS}}^i, \text{Pf}_{\text{ID}_j}^i, \sigma_{\text{SPHINCS+C}}^i);$

Algorithm 2: Verify: Verification Procedure

Input: $\text{gpk}_G, pw_{\text{ID}_j}^{i,z}, T, \text{RLG}$
Output: 1 if verification succeeds, 0 otherwise

/ Step 1: Revocation check */*

1 **if** $C_{\text{ID}_j}^i \notin \{RS_{s_j}^i\}_{j \in [U]}$ **then**
2 **return** 0; // Member not valid or revoked

3 Find idx such that RS_{idx} contains $C_{\text{ID}_j}^i$;

/ Step 2: Compute updated verification point */*

4 $vp_{\text{ID}_j}^i \leftarrow$ from $\bar{pw}_{\text{ID}_j}^{i,z}$ and T ;
5 $\bar{vp}_{\text{ID}_j}^i \leftarrow H(vp_{\text{ID}_j}^i \| C_{\text{ID}_j}^i \| i)$;

/ Step 3: Reconstruct OTS-tree root */*

6 $Rt \leftarrow$ derived from $pk_{idx}^i, P_{\text{ID}_j}^i$;

/ Step 4: Verify TOTP password and signatures */*

7 **if** $\text{TOTP.Verify}(vp_{\text{ID}_j}^i, \bar{pw}_{\text{ID}_j}^{i,z}, T) = 1$ **and** $\text{OTS.Verify}(pk_{idx}^i, \bar{vp}_{\text{ID}_j}^i, \sigma_{\text{OTS}}^i) = 1$
 and $\text{SPHINCS+C.Verify}(\text{gpk}_G, Rt, \sigma_{\text{SPHINCS+C}}^i) = 1$ **then**
8 **return** 1; // All checks succeed

9 **else**
10 **return** 0; // Reject if any check fails

D Proof of Theorem 1

We establish the claim using a sequence-of-games argument, starting from the original **Anony** experiment and step by step modifying it until the adversary's advantage disappears. For each game i , let $\text{BK}_i^{\text{Anony}}$ denote the event that the adversary wins (i.e., the experiment outputs 1).

Game 0. This game is the real **Anony** game. We have that

$$\Pr[\text{BK}_0^{\text{Anony}}] = 1/2 + \text{Adv}_{\mathcal{A}, \text{LWDGT}}^{\text{Anony}}(\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e).$$

Game 1. The challenger now aborts if two independent runs of PRF.Setup produce the same key. Such collisions occur only with negligible probability, otherwise one could directly break PRF security. Concretely, suppose the PRF challenger provides a challenge key k^* , while the adversary samples $U - 1$ additional keys. If any two coincide, then with probability $2/U$ the challenge key is among them, revealing k^* and violating the PRF assumption. Therefore,

$$\Pr[\text{BK}_0^{\text{Anony}}] \leq \Pr[\text{BK}_1^{\text{Anony}}] + \frac{U}{2} \cdot \text{Adv}_{\mathcal{A}, \text{PRF}}^{\text{PRF}}(\kappa, 1).$$

Game 2. We change this game from the previous game by adding one abort condition. Namely, the challenger additionally aborts if it fails to correctly identify the two honest members (out of U) and the challenge verification epoch (out of

E) used in the **Challenge** query. The success probability is bounded by $1/(2U \cdot E)$. Hence,

$$\Pr[\text{BK}_1^{\text{Anony}}] = 2U \cdot E \cdot \Pr[\text{BK}_2^{\text{Anony}}].$$

Game 3. In this game, the values derived from the PRF at the guessed challenge epoch i^* , namely the set $\{k_{\text{PM}}^{i^*}, k_{\text{ASE}}^{i^*}, sd_{\alpha_{\text{ID}_0}}^{i^*}, sd_{\alpha_{\text{ID}_1}}^{i^*}\}$, are replaced by independent, uniformly random strings of the same length. The rest of the experiment remains unchanged. To argue indistinguishability, we use a hybrid argument defining a sequence of experiments H_0, H_1, H_2, H_3, H_4 , where H_0 corresponds to the previous game and H_4 to the current game. Each subsequent hybrid replaces one additional element from the set by a random string. If an adversary $\mathcal{D}$ distinguishes between two consecutive hybrids H_{j-1} and H_j , we build a reduction $\mathcal{B}_j$ that breaks the security of the underlying PRF. The reduction $\mathcal{B}_j$ uses oracle access to a challenger which returns either a real PRF evaluation or a random string on a query label. By embedding this oracle response in the position of the j -th replaced element for epoch i^* , $\mathcal{B}_j$ perfectly simulates either H_{j-1} or H_j . Summing over all four replacements, the total distinguishing advantage is bounded by four times the PRF advantage, yielding

$$\Pr[\text{BK}_2^{\text{Anony}}] \leq \Pr[\text{BK}_3^{\text{Anony}}] + 4 \cdot \text{Adv}_{\mathcal{A}, \text{PRF}}^{\text{PRF}}(\kappa, E).$$

Game 4. The game proceeds exactly as the previous game, but the challenge ciphertext produced during the **Challenge** query is always generated using ID_0 , regardless of the challenge bit b . If an adversary $\mathcal{A}$ can tell this game apart from the previous one, then we can construct an algorithm $\mathcal{C}$ that breaks the IND-CCA security of ASE. The reduction works as follows: the adversary submits the pair $(\alpha_{\text{ID}_0}, \alpha_{\text{ID}_1})$ to the ASE challenger and receives the challenge ciphertext. If the ASE challenger encrypted α_{ID_0} , then the distribution matches the previous game; if it encrypted α_{ID_1} , then the distribution matches the current game. Thus, distinguishing between the two games translates into an IND-CCA advantage against ASE. Hence,

$$\Pr[\text{BK}_3^{\text{Anony}}] \leq \Pr[\text{BK}_4^{\text{Anony}}] + \text{Adv}_{\mathcal{A}, \text{ASE}}^{\text{IND-CCA}}(\kappa, E).$$

The changed game implies that the adversary cannot infer any information regarding the identity from the ASE ciphertext.

Game 5. Finally, instead of relying on the permutation mechanism, the challenger simply chooses uniformly at random between the two leaves intended for ID_0 and ID_1 . An adversary that can detect this change would break the unpredictability of PM. In more detail, the reduction can query the PM challenger on the set $\{1, \dots, U\}$ with two positions corresponding to the challenge members' join order. If the output of PM aligns with the chosen leaf, then the simulated game is the same as before; otherwise it corresponds to the modified game. The adversary's decision can then be used to predict the output of PM, violating its assumed security. We therefore conclude that

$$\Pr[\text{BK}_4^{\text{Anony}}] \leq \Pr[\text{BK}_5^{\text{Anony}}] + \text{Adv}_{\mathcal{A}, \text{PM}}^{\text{UP}}(\kappa).$$

In this last game, the adversary learns nothing about which identity corresponds to which verification point. Even in the worst case, where all group members except ID_0 and ID_1 are malicious, the two honest members still occupy two indistinguishable positions chosen at random. Thus the adversary's success probability is exactly $1/2$, i.e., $\Pr[\text{BK}_5^{\text{Anony}}] = \frac{1}{2}$.

Collecting the bounds over all transitions, the adversary's overall advantage is negligible. This completes the proof of Theorem 1.

E Proof of Theorem 2

Let us define $\text{BK}_i^{\text{Trace}}$ as the event that the i -th instance of the (modified) Trace game returns 1.

Game 0. This is the original Trace game. By definition, we have

$$\Pr[\text{BK}_0^{\text{Trace}}] = \text{Adv}_{\mathcal{A}, \text{LWDGT}}^{\text{Trace}}(\kappa, G, \Delta_{pw}, \Delta_{vp}, T_s, T_e).$$

Game 1. In this game, we proceed as in the previous one, except that the challenger aborts if any two honest group members happen to share the same PRF key. Using an argument analogous to Game 1 in the proof of Theorem 1, we obtain

$$\Pr[\text{BK}_0^{\text{Trace}}] \leq \Pr[\text{BK}_1^{\text{Trace}}] + \frac{U}{2} \cdot \text{Adv}_{\mathcal{A}, \text{F}}^{\text{PRF}}(\kappa, 1).$$

Game 2. Let vp^* be the verification point corresponding to the password pw^* submitted in the Finalize query. In this game, the challenger additionally aborts if there exist indices i^* , u such that $H(vp^* || C_{\text{ID}_j^*} || i^*) = H(vp_{\text{ID}_j}^u || C_{\text{ID}_j}^u || u)$ and ID^* is registered by AddHM. If an adversary triggers this abort with non-negligible probability, one can construct an algorithm that breaks the collision resistance of H . Therefore, we have

$$\Pr[\text{BK}_1^{\text{Trace}}] \leq \Pr[\text{BK}_2^{\text{Trace}}] + \text{Adv}_{\mathcal{A}, \text{H}}^{\text{CRH}}(\kappa).$$

Game 3. This game is identical to the previous one, except that the challenger aborts if the adversary outputs a password $pw^* = (pw_{\text{ID}_j}^*, C_{\text{ID}^*}^i, \sigma_{\text{ID}^*}^i)$ for some T^* such that: i) $\text{Verify}(\text{gpk}_G, pw_{\text{ID}_j}^*, T^*, \text{RL}_G) = 1$; and ii) $\sigma_{\text{ID}^*}^i$ was not produced by the challenger during AddHM or AddMM. If this event occurs with non-negligible probability, it can be used to break the EU-CMA security of the SPHINCS+C, OTS and MT schemes. Hence,

$$\Pr[\text{BK}_2^{\text{Trace}}] \leq \Pr[\text{BK}_3^{\text{Trace}}] + \text{Adv}_{\mathcal{A}, \text{SPHINCS+C}}^{\text{EU-CMA}}(\kappa, E) + \text{Adv}_{\mathcal{A}, \text{OTS}}^{\text{EU-CMA}}(\kappa, 1) + \text{Adv}_{\mathcal{A}, \text{MT}}^{\text{MT-Forge}}(\kappa).$$

Game 4. In this step, the challenger additionally aborts if it cannot correctly guess the owner ID^* and the verification epoch i^* of the forged password. The probability of a correct guess is at most $1/(U \cdot E)$, leading to $\Pr[\text{BK}_3^{\text{Trace}}] = U \cdot E \cdot \Pr[\text{BK}_4^{\text{Trace}}]$. We let the i^* -th epoch be the guessed verification epoch, and denote the target uncorrupted group member as ID^* .

Game 5. Here, the outputs of F corresponding to $\{k_{\text{PM}}^{i*}, k_{\text{ASE}}^{i*}, sd_{\alpha_{\text{ID}_0}}^{i*}, sd_{\alpha_{\text{ID}_1}}^{i*}\}$ are replaced with truly random values. By the same reasoning as in Game 3 of Theorem 1, we obtain

$$\Pr[\text{BK}_4^{\text{Trace}}] \leq \Pr[\text{BK}_5^{\text{Trace}}] + 4 \cdot \text{Adv}_{\mathcal{A}, F_1}^{\text{PRF}}(\kappa, E).$$

Game 6. Finally, the challenger aborts if the adversary outputs a valid password $\bar{pw}^*$ for ID^* at some T^* that is later than all previously generated password values. Any such output could be leveraged to break TOTP security. Constructing an algorithm $\mathcal{B}$ that simulates this modified **Trace** game while querying $\text{GetNextPw}()$ appropriately yields

$$\Pr[\text{BK}_5^{\text{Trace}}] \leq \Pr[\text{BK}_6^{\text{Trace}}] + \text{Adv}_{\mathcal{A}, \text{TOTP}}^{\text{TOTP_Forge}}(\kappa, T_s, T_e, \Delta_{pw}).$$

At this stage, the adversary cannot manipulate any uncorrupted member's password values, implying that the advantage is 0. This completes the proof.